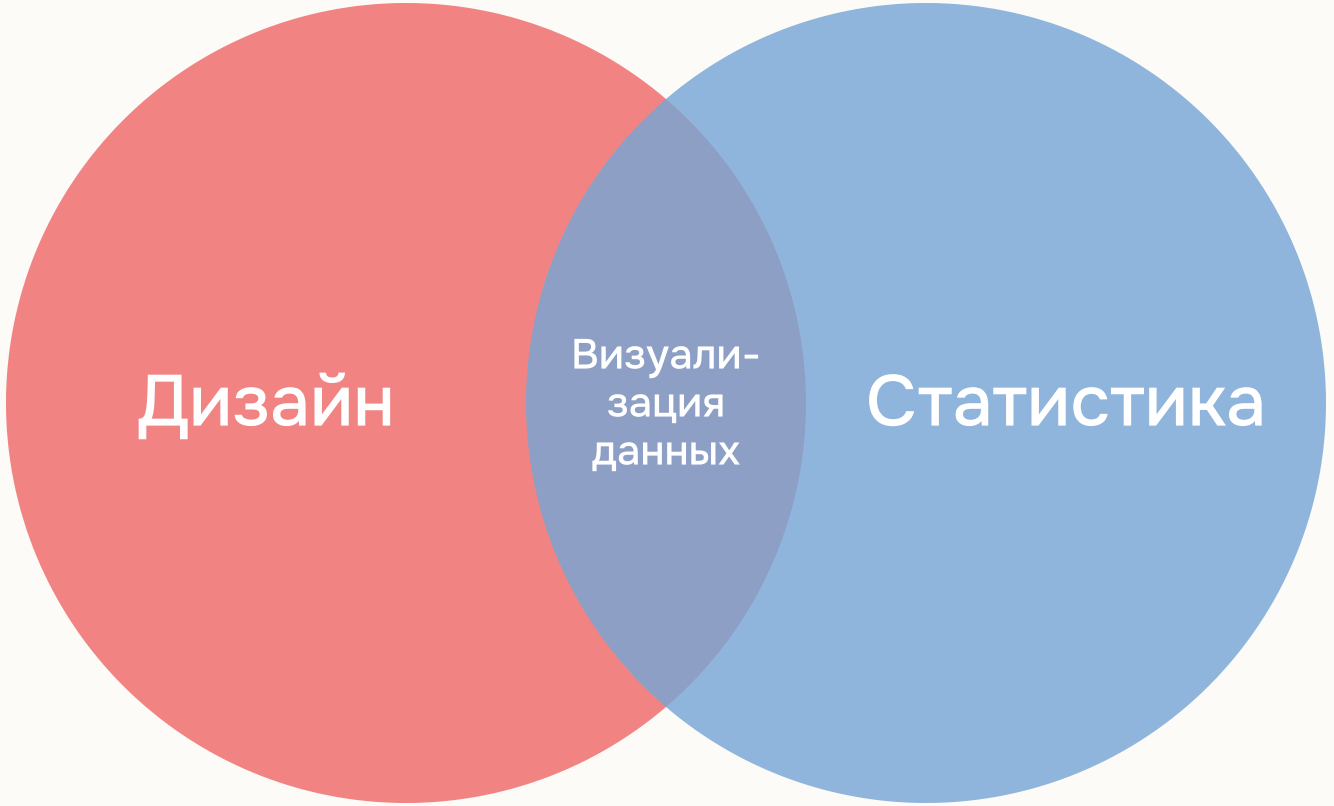
Введение в ggplot2

Грамматика графики · Столбцы, линии и темы

Подготовил Давид Кристофер

Визуализация данных и наука о данных

Ключевой навык в науке о данных





Исследовательская визуализация: быстрые, насыщенные данными графики для специалистов, которые ведут анализ.

Объясняющая визуализация: отточенные, сфокусированные графики для широкой аудитории.

```
head(MASS::mammals, 15)
```

```
                         body  brain
Arctic fox              3.385  44.50
Owl monkey              0.480  15.50
Mountain beaver         1.350   8.10
Cow                   465.000 423.00
Grey wolf              36.330 119.50
Goat                   27.660 115.00
Roe deer               14.830  98.20
Guinea pig              1.040   5.50
Verbet                  4.190  58.00
Chinchilla              0.425   6.40
Ground squirrel         0.101   4.00
Arctic ground squirrel  0.920   5.70
African giant pouched rat   1.000   6.60
Lesser short-tailed shrew   0.005   0.14
Star-nosed mole         0.060   1.00
```

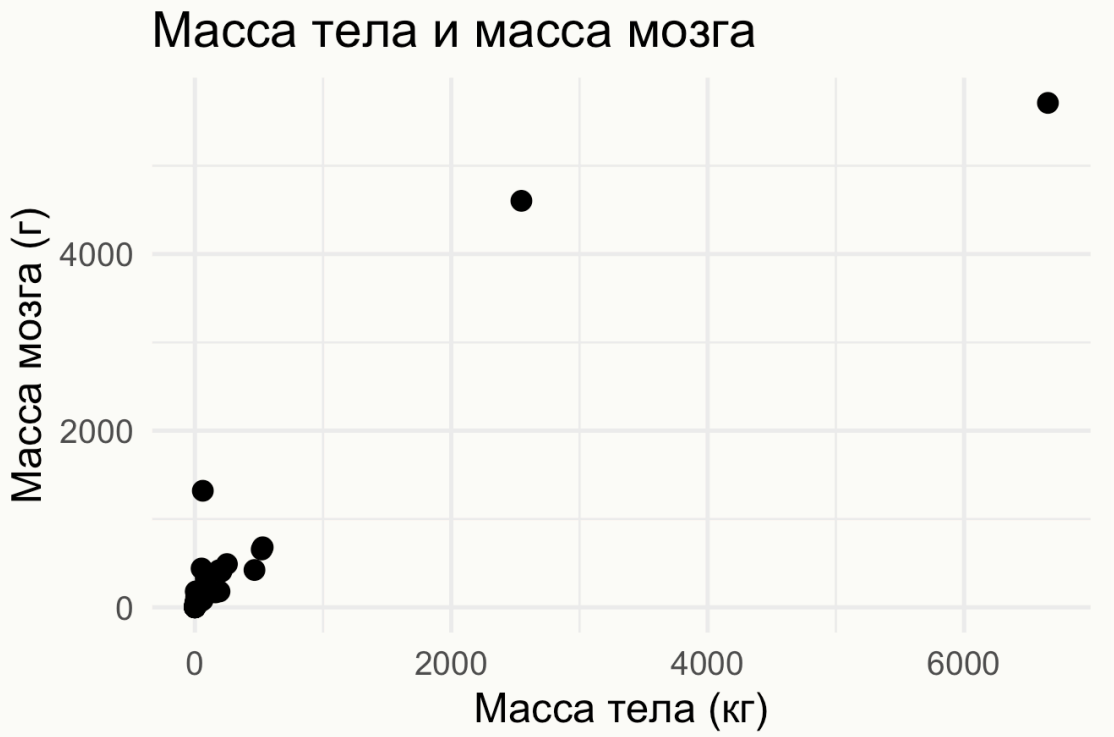
Набор данных содержит средние массы мозга и тела 62 наземных млекопитающих.

Диаграмма рассеяния

```
library(ggplot2)

mammals = MASS::mammals

ggplot(mammals, aes(x = body, y = brain)) +
  geom_point(size = 3) +
  theme_minimal(base_size = 16) +
  labs(
    title = "Масса тела и масса мозга",
    x = "Масса тела (кг)",
    y = "Масса мозга (г)"
  )
```



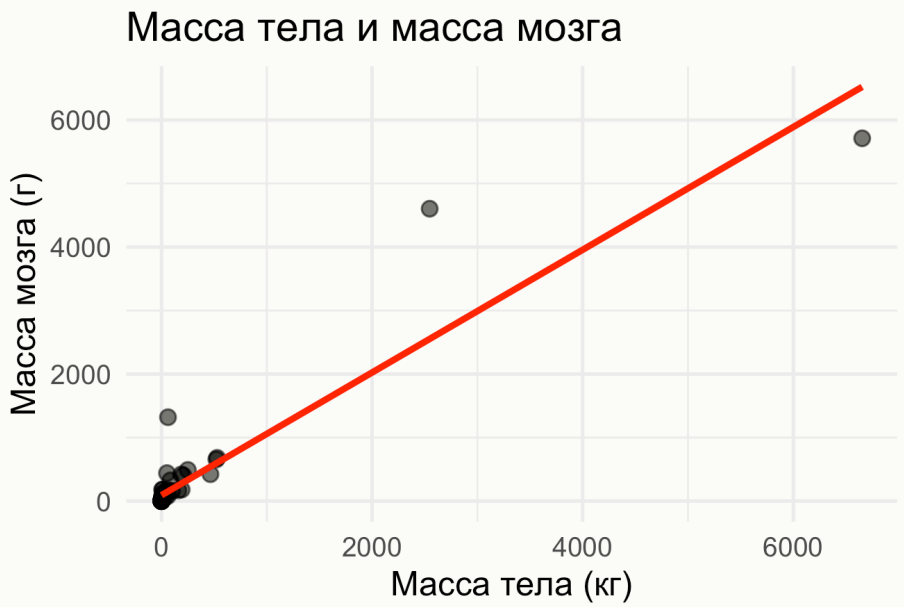
У двух млекопитающих — африканского и азиатского слонов — очень большая масса и тела, и мозга.

Исследуем с помощью линейной модели

```
library(ggplot2)

mammals = MASS::mammals

ggplot(mammals, aes(x = body, y = brain)) +
  geom_point(size = 3, alpha = 0.6) +
  theme_minimal(base_size = 16) +
  labs(
    title = "Масса тела и масса мозга",
    x = "Масса тела (кг)",
    y = "Масса мозга (г)"
  ) +
  stat_smooth(
    method = "lm",
    color = "red",
    se = FALSE
  )
```



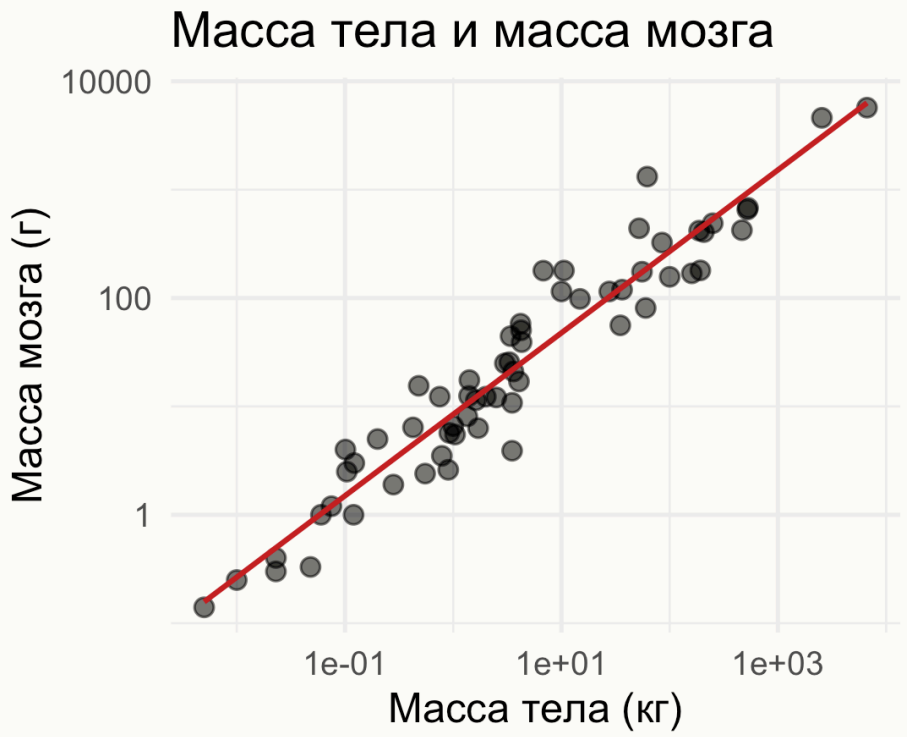
- Несколько экстремальных значений сильно влияют на модель
- В этом случае линейная модель — неудачный выбор

Исследование: точная настройка

```
library(ggplot2)

mammals = MASS::mammals

ggplot(mammals, aes(x = body, y = brain)) +
  geom_point(size = 3, alpha = 0.6) +
  theme_minimal(base_size = 16) +
  coord_fixed() +
  scale_x_log10() +
  scale_y_log10() +
  labs(
    title = "Масса тела и масса мозга",
    x = "Масса тела (кг)",
    y = "Масса мозга (г)"
  ) +
  stat_smooth(
    method = "lm",
    color = "#C42126",
    se = FALSE,
    linewidth = 1
  )
```



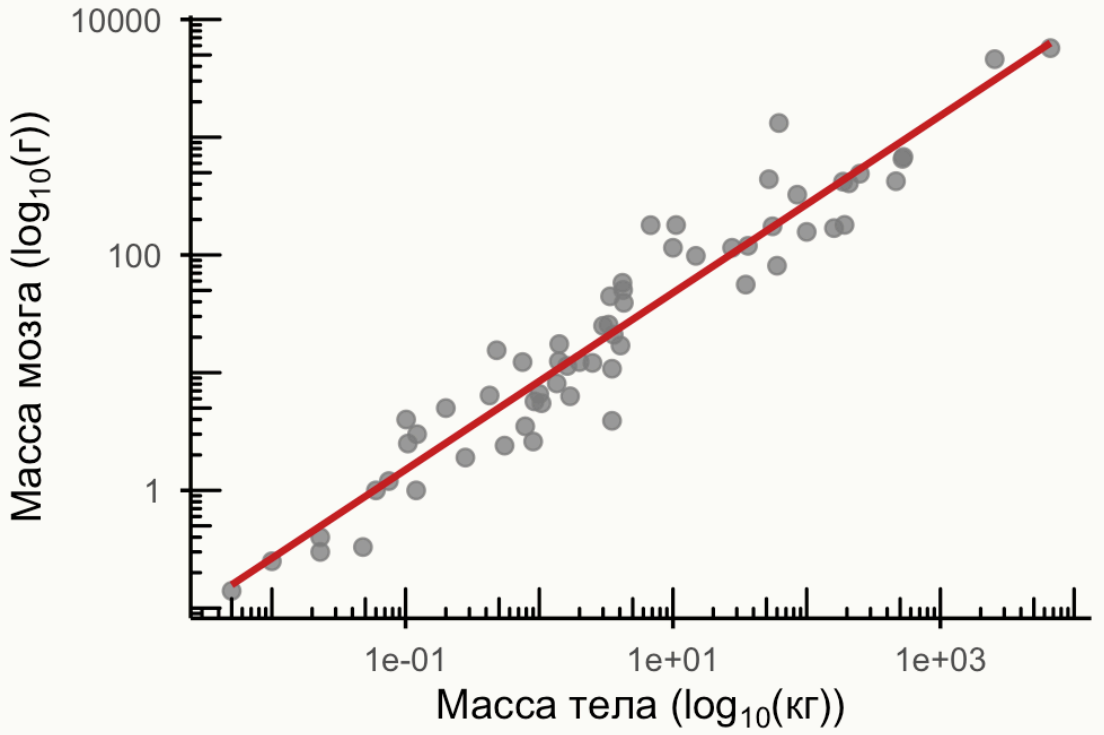
После логарифмирования обеих переменных прямая гораздо лучше описывает данные.

В итоге нам нужен аккуратный объясняющий график для публикации

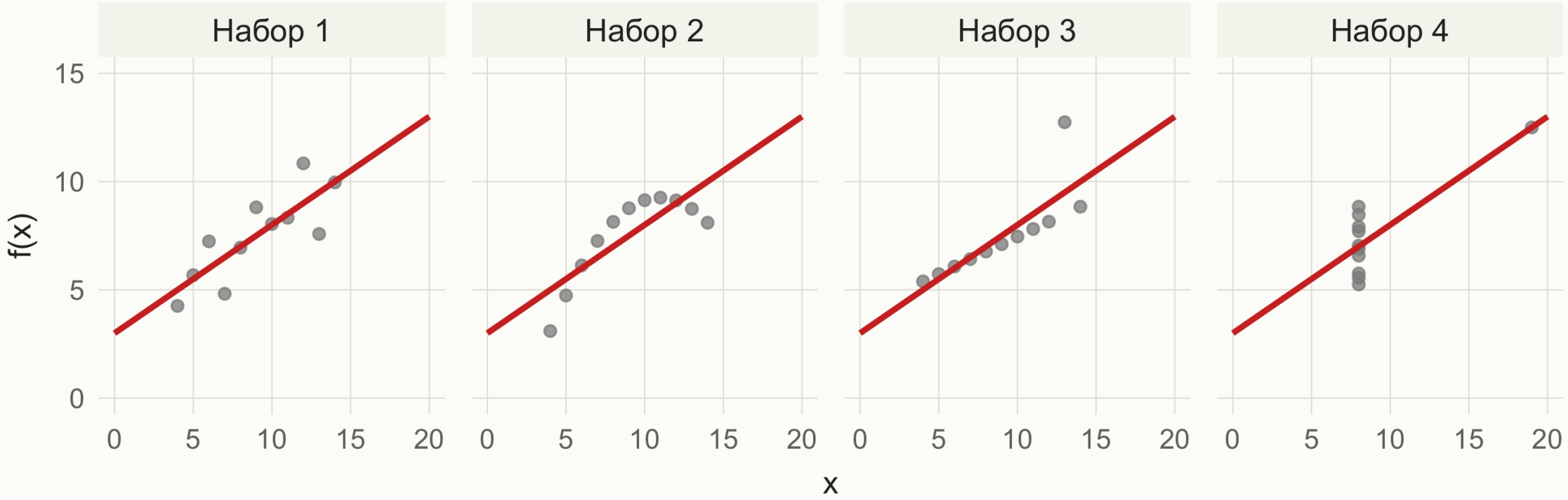
```
library(ggplot2)

mammals = MASS::mammals

ggplot(mammals, aes(x = body, y = brain)) +
  theme_minimal(base_size = 12) +
  geom_point(color = "gray50", size = 2, alpha = 0.8) +
  geom_smooth(
    method = "lm",
    color = "#C42126",
    se = FALSE,
    linewidth = 1
  ) +
  scale_x_log10(
    name = expression("Масса тела ("*log[10]*"(кг))")
  ) +
  scale_y_log10(
    name = expression("Масса мозга ("*log[10]*"(г))")
  ) +
  theme(
    panel.grid = element_blank(),
    axis.line = element_line(color = "black"),
    axis.ticks = element_line(color = "black")
  ) +
  annotation_logticks()
```



Классический пример Фрэнсиса Энскомба, впервые опубликованный в 1973 году.



Если бы мы опирались только на числовые результаты линейной модели и не построили данные, мы пропустили бы разные интересные закономерности.

У всех четырёх наборов почти одинаковые средние x и y, дисперсии, корреляция и линия регрессии.

Грамматика графики

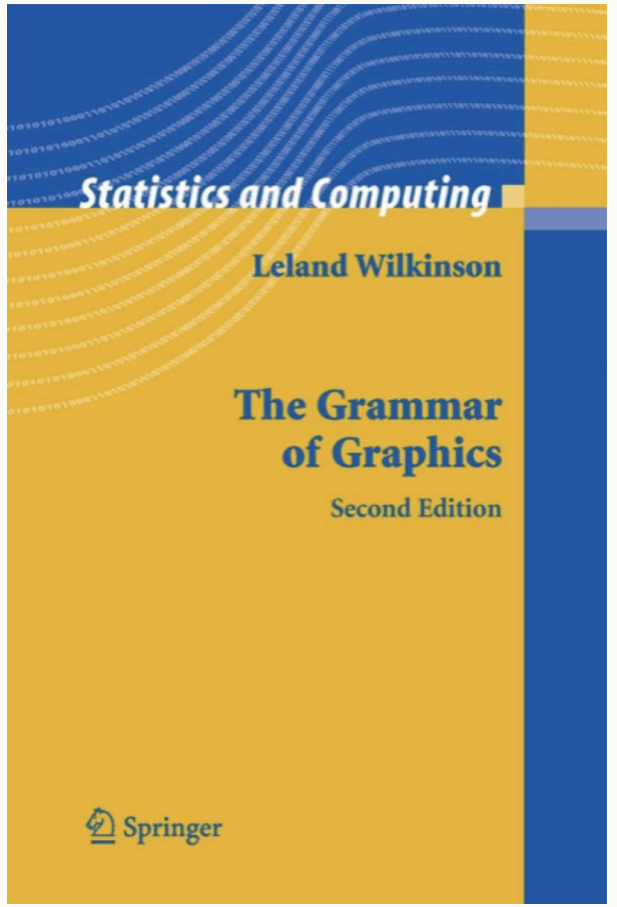
Быстрая рыжая лиса перепрыгивает через ленивую собаку

Прилагательное	*Быстрая рыжая*	*Бешеная рыжая*	
Существительное	*лиса*	*лиса*	*Охотник*
Глагол	*перепрыгивает*	*укусила*	*застрелил*
Предлог	*через*		
Прилагательное	*ленивую*	*дружелюбную*	*бешеную рыжую*
Существительное	*собаку.*	*собаку.*	*лису.*

Каждое слово в предложении играет свою грамматическую роль. Изменение любого элемента может слегка или кардинально изменить смысл.

Грамматика графики

- Графики строятся по структуре, похожей на грамматику языка
- Основана на книге Уилкинсона «The Grammar of Graphics» (1999)
- Состоит из отдельных слоёв — визуального словаря
- Слои подобны прилагательным и существительным
- Эстетические отображения подобны грамматическим правилам



Три основных грамматических элемента

Элемент	Описание
Данные	Набор данных, который мы изображаем
Эстетики	Шкалы, на которые мы *отображаем* данные
Геометрии	Визуальные элементы, которыми изображаем данные

Семь грамматических элементов

Элемент	Описание
Данные	Набор данных, который мы изображаем
Эстетики	Шкалы, на которые мы *отображаем* данные
Геометрии	Визуальные элементы, которыми изображаем данные
Темы	Всё оформление, которое не является данными
Статистика	Вычисления по данным: подсчёты, средние, сглаживание
Координаты	Пространство, в котором строится график
Фасеты	Разбиение графика на панели по группам

Данные data	*{интересующие нас переменные}* {variables of interest}				
Эстетики aesthetics	*ось x* *ось y* x, y	*цвет* *заливка* color, fill	*размер* *текст* size, label	*прозрачность* *форма* alpha, shape	*толщина линии* *тип линии* linewidth, linetype
Геометрии geometries	*точка* point	*линия* line	*гистограмма* histogram	*столбец* bar	*ящик с усами* boxplot
Темы themes	*всё, кроме данных* non-data ink				
Статистика statistics	*интервалы* binning	*сглаживание* smoothing	*описательные статистики* descriptive	*статистический вывод* inferential	
Координаты coordinates	*декартовы* cartesian	*фиксированные* fixed	*полярные* polar	*пределы* limits	
Фасеты facets	*столбцы* columns	*строки* rows			

Слои ggplot2

Данные



Набор данных iris

Setosa



Versicolor



Virginica



1. Fisher, R. A. (1936) The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7, Part II, 179–188.
2. Anderson, Edgar (1935). The irises of the Gaspe Peninsula. *Bulletin of the American Iris Society*, 59, 2–5.

Набор данных iris

```
head(iris)
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
```

Эстетики



Эстетики для iris

Species	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
	X	Y		

Мы отобразим `Sepal.Length` на эстетику X, а `Sepal.Width` — на эстетику Y.

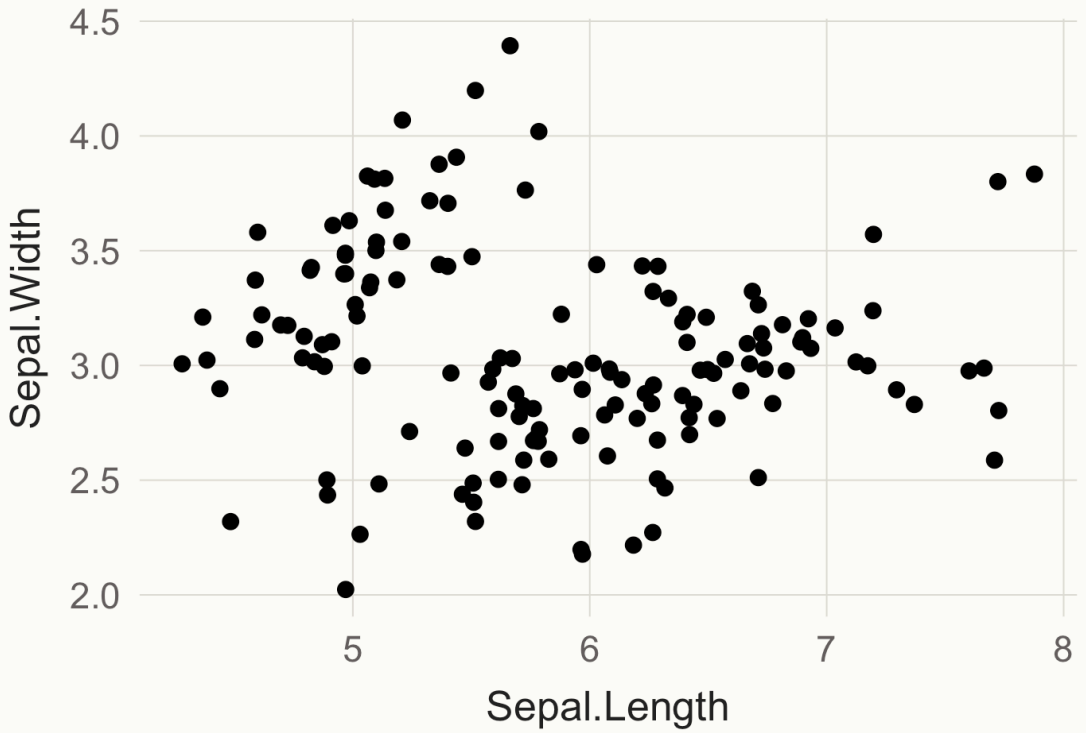
Геометрии



Геометрии для iris

```
g <- ggplot(iris,
  aes(
   x = Sepal.Length,
   y = Sepal.Width
  )) +
  geom_jitter()

g
```

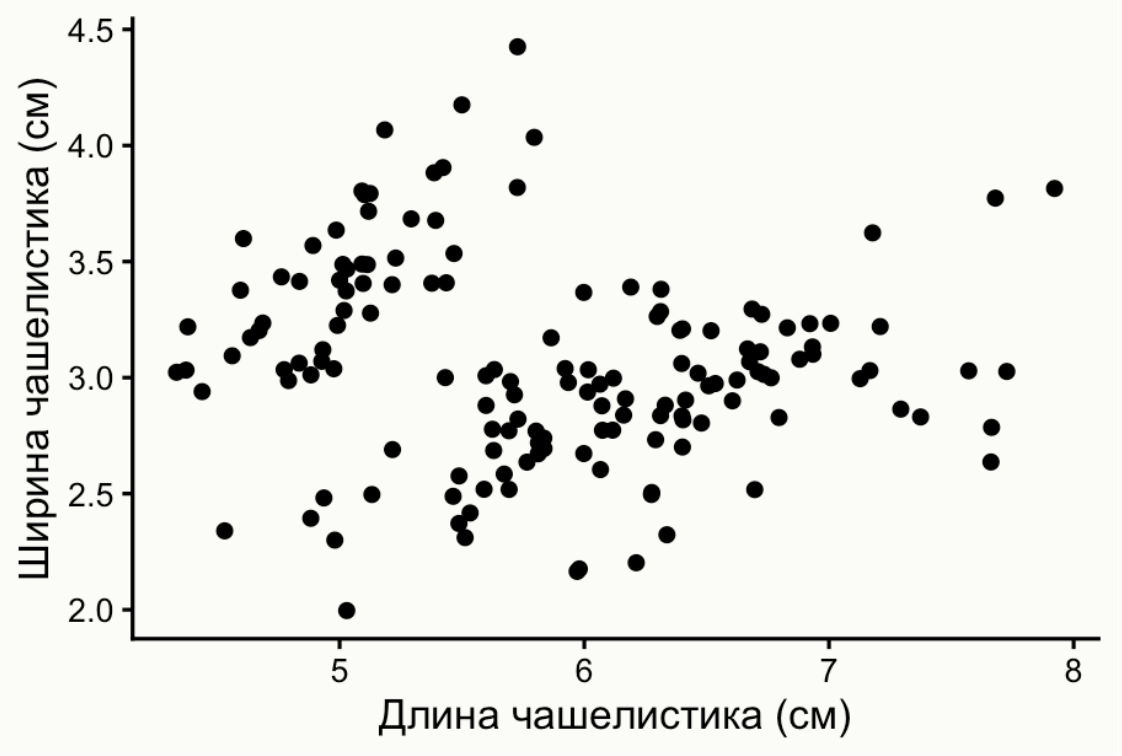


Темы



Темы для iris

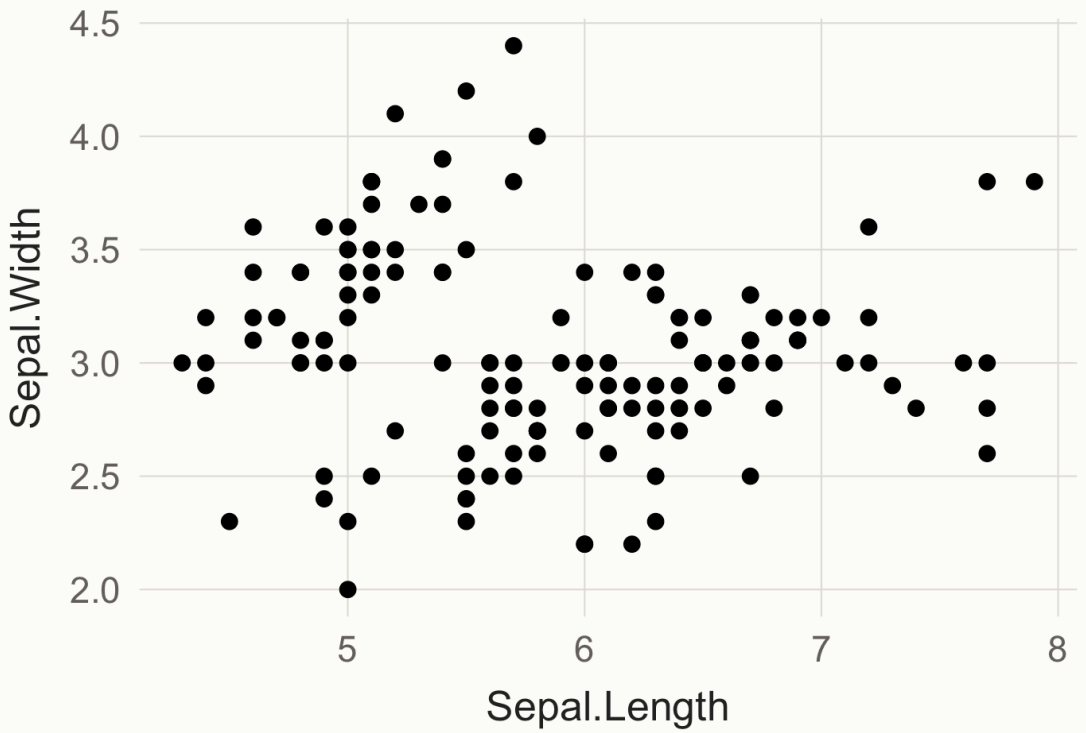
```
g <- g +
  labs(
    x = "Длина чашелистика (см)",
    y = "Ширина чашелистика (см)"
  ) +
  theme_classic()
g
```



Эстетики

Отображение на оси X и Y

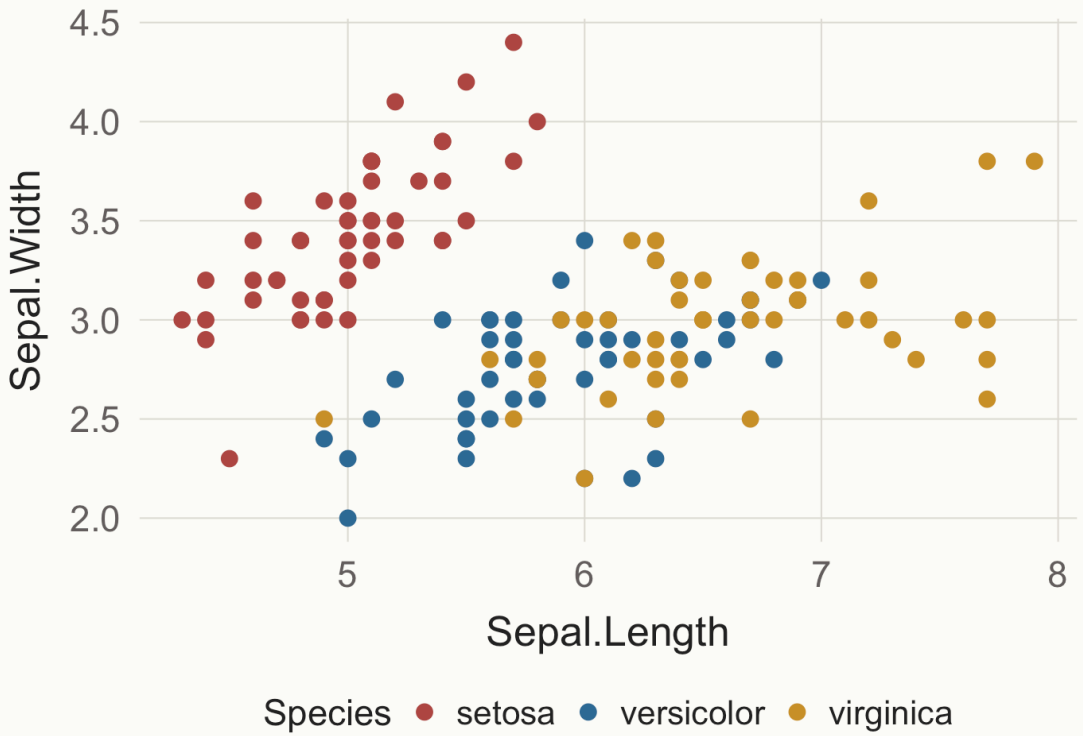
```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width)) +
  geom_point()
```



Отображение на цвет

```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_point()
```

Тип	Переменная
Цвет	Species



Species, столбец таблицы, *отображается* **на цвет** — видимую эстетику.

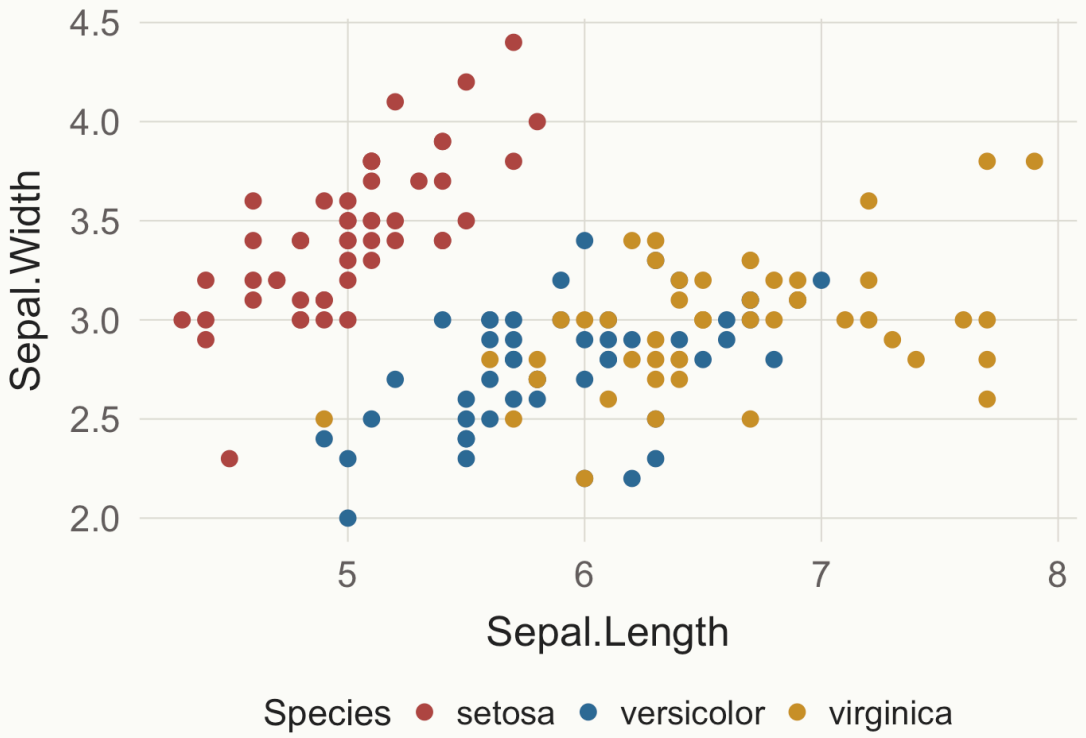
Отображения задают в `aes()`.

Отображение цвета внутри геометрии

```
ggplot(iris) +
  geom_point(aes(x = Sepal.Length,
                 y = Sepal.Width,
                 col = Species))
```

`aes()` **внутри геометрии нужен, только если:**

- Другие слои не должны наследовать эти эстетики.
- У слоёв разные данные.



Типичные видимые эстетики

Эстетика	Описание
x	Положение по оси X
y	Положение по оси Y
fill	Цвет заливки
color	Цвет точек, контуров других геометрий
size	Площадь или радиус точек, толщина линий

Эстетика	Описание
alpha	Прозрачность
linetype	Тип линии: сплошная, пунктир...
labels	Текст на графике или осях
shape	Форма

Эстетику можно связать с переменной в `aes()` или задать ей одно значение для всех точек в `geom_*()`. Не задавайте одну эстетику обоими способами сразу: значение из `geom_*()` перекроет отображение.

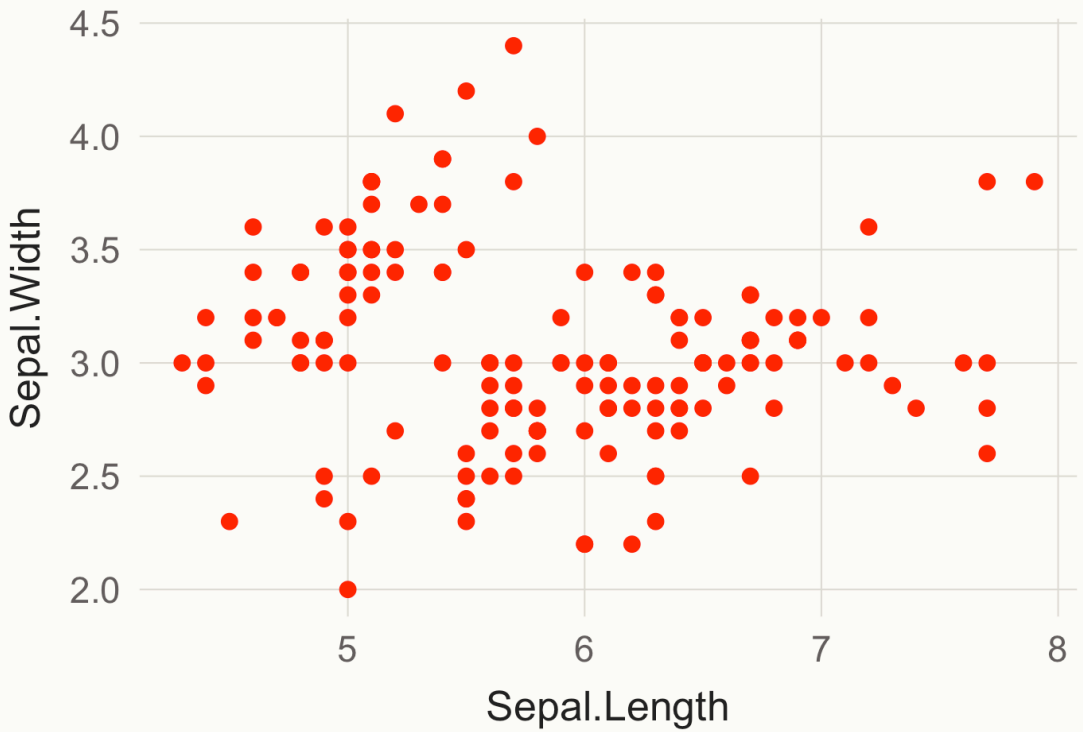
Эстетики? Атрибуты!

```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width)) +
  geom_point(color = "red")
```

Тип	Свойство
Цвет	"red"

Атрибуты задают в `geom_*()`

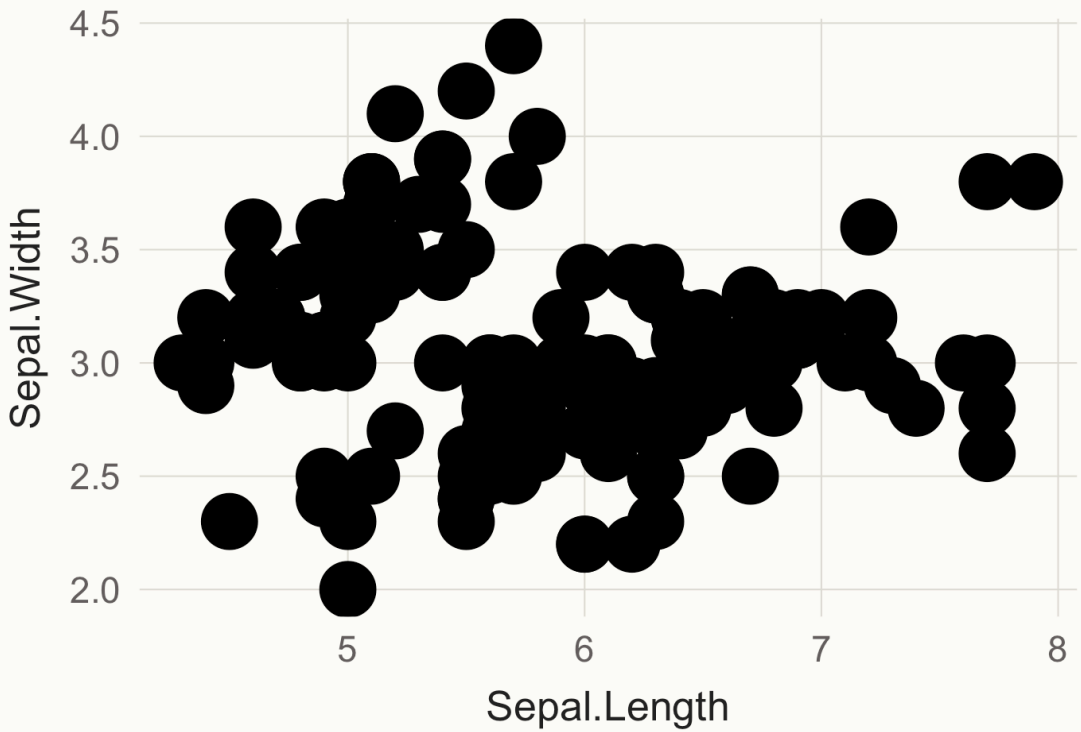
Атрибуту **color** *присвоено* значение **"red"**.



Эстетики? Атрибуты!

```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width)) +
  geom_point(size = 10)
```

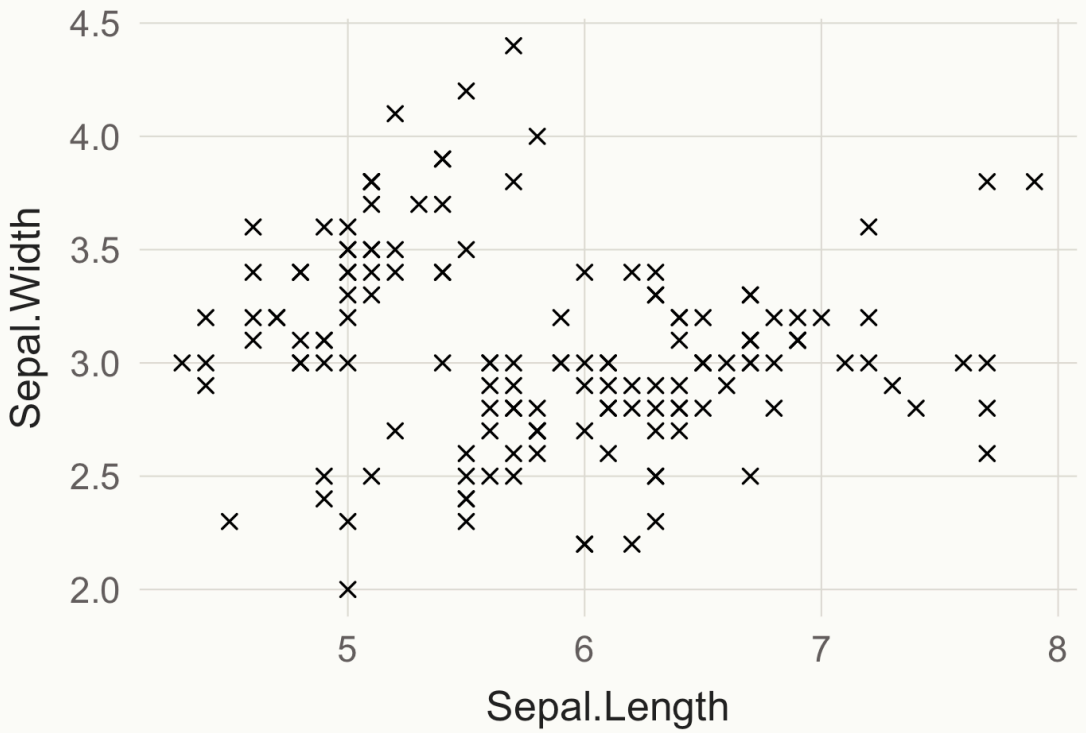
Тип	Свойство
Размер	10



Эстетики? Атрибуты!

```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width)) +
  geom_point(shape = 4)
```

Тип	Свойство
Форма	4



Изменение эстетик

Позиции

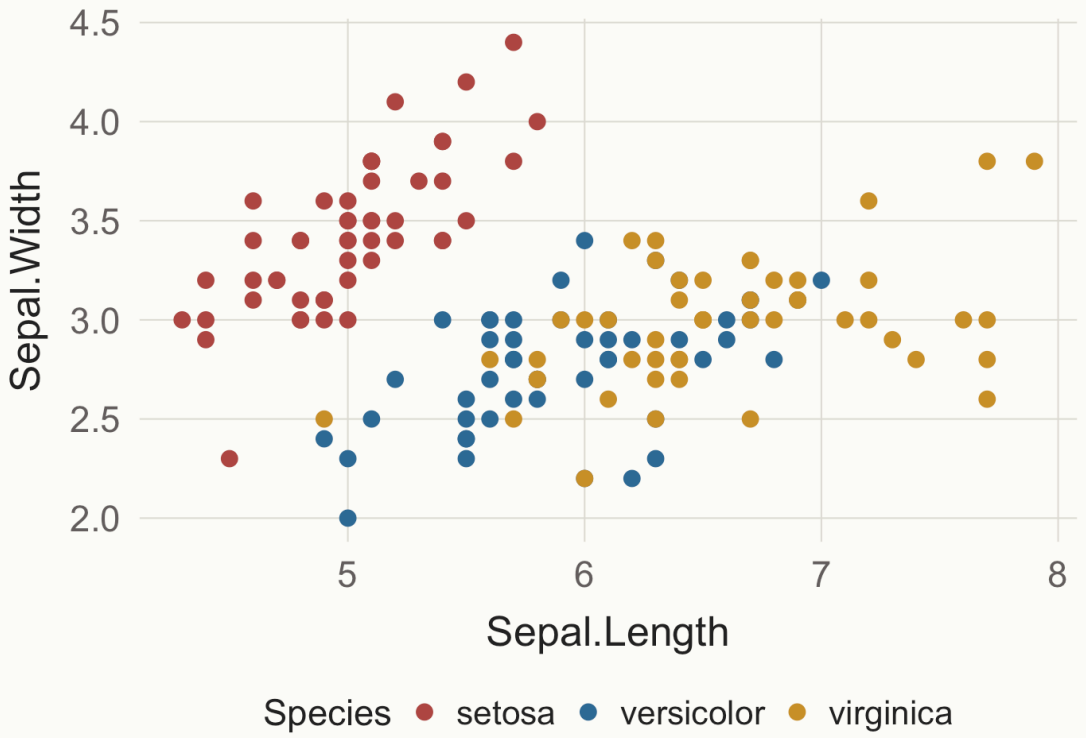
Как располагать перекрывающиеся объекты

- identity
- dodge
- stack
- fill
- jitter
- jitterdodge
- nudge

position = "identity" (по умолчанию)

```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_point()
```

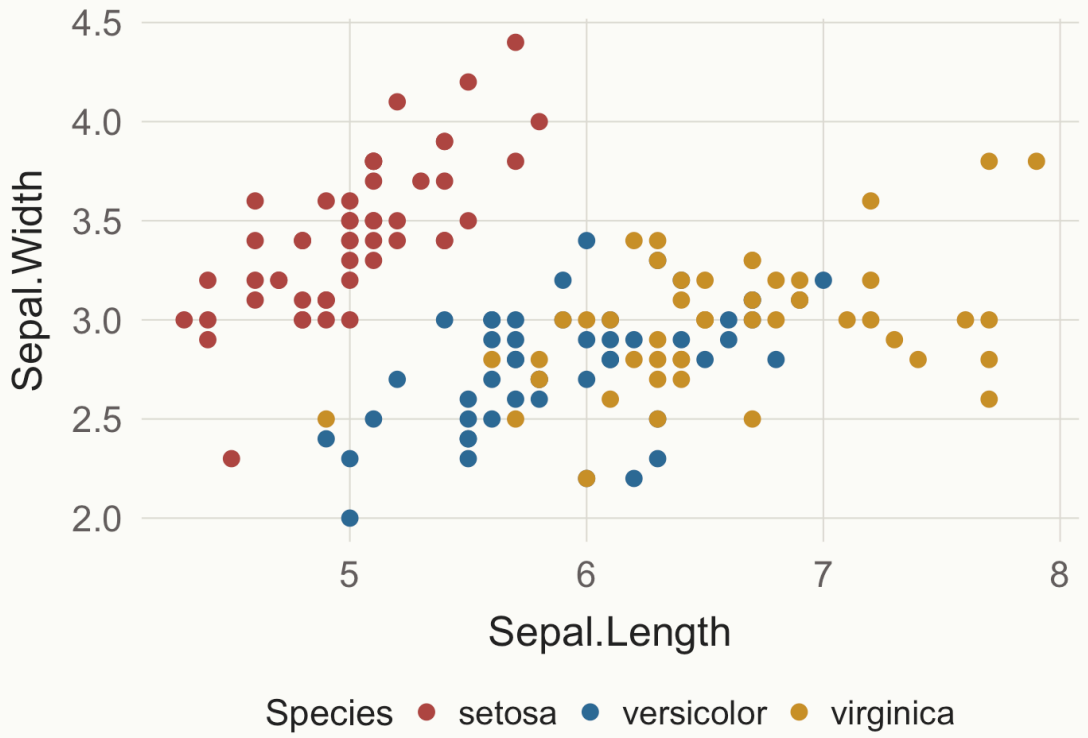
«identity» — точки стоят ровно там, где задают значения из данных, без сдвига.



position = "identity" (по умолчанию)

```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_point(position = "identity")
```

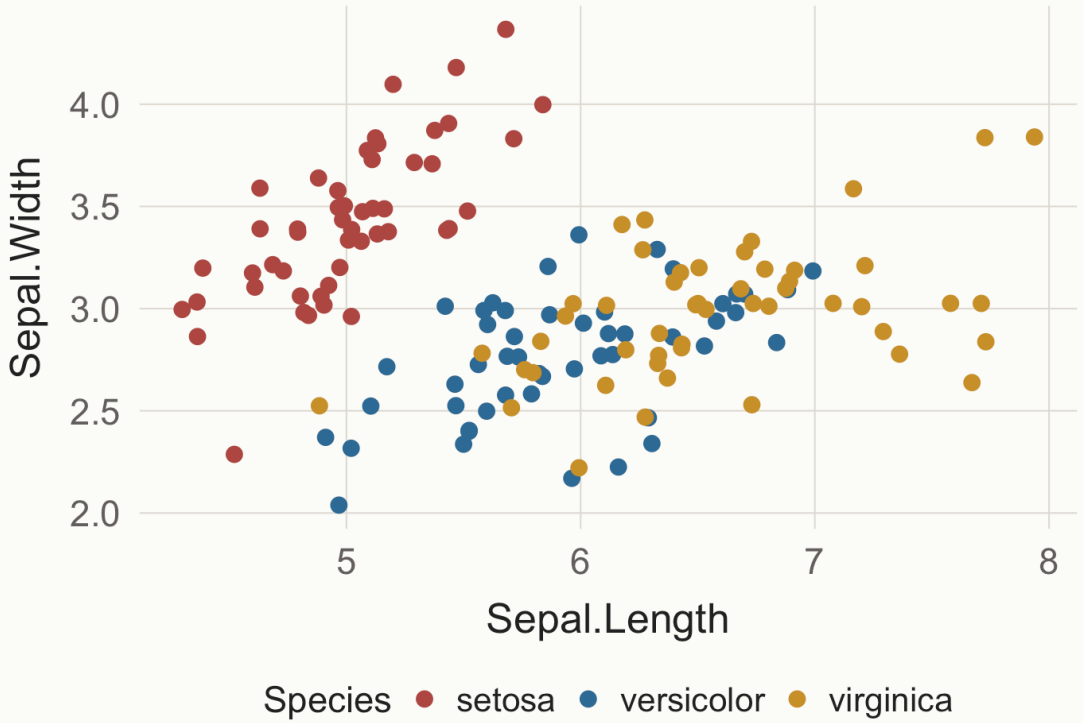
«identity» — точки стоят ровно там, где задают значения из данных, без сдвига.



position = "jitter"

```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_point(position = "jitter")
```

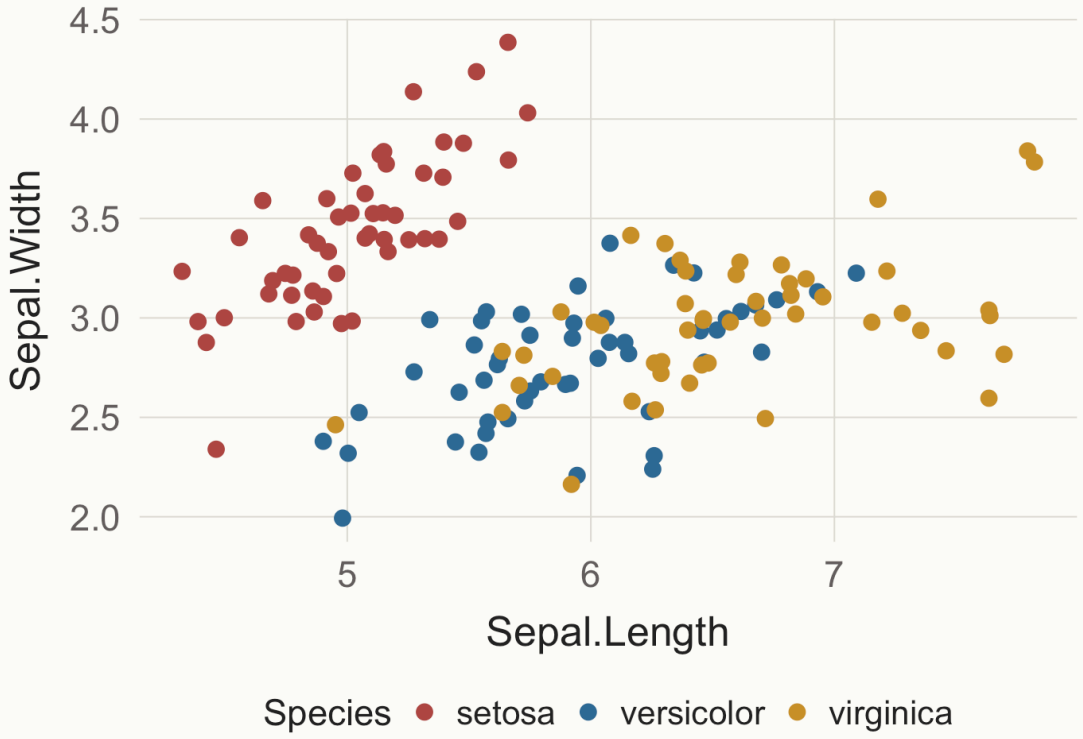
- Как ни странно, добавление случайного шума иногда делает график понятнее.
- Jitter особенно полезен для небольших наборов данных.
- Позицию можно задать строкой `"jitter"` или функцией `position_jitter()` с настройками.



`position_jitter()`

```
# 0.1 — ширина случайного сдвига (width)
posn_j <- position_jitter(0.1)

ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 col = Species)) +
  geom_point(position = posn_j)
```



Преимущества:

- Можно задать аргументы позиции
- Один объект `posn_j` можно использовать в нескольких графиках и слоях

Функции шкал

Вспомним: каждая эстетика — это шкала, на которую мы отображаем данные. Цвет — такая же шкала, как x и y.

Ко всем шкалам можно обратиться через функции `scale_`:

- `scale_x_*()`
- `scale_y_*()`
- `scale_color_*()`
  - также `scale_colour_*()`
- `scale_fill_*()`
- `scale_shape_*()`
- `scale_linetype_*()`
- `scale_size_*()`

У каждой эстетики, которую мы видели, есть своя функция шкалы.

Функции шкал

Третья часть имени должна соответствовать типу данных. Здесь discrete означает, что мы работаем с категориальными данными:

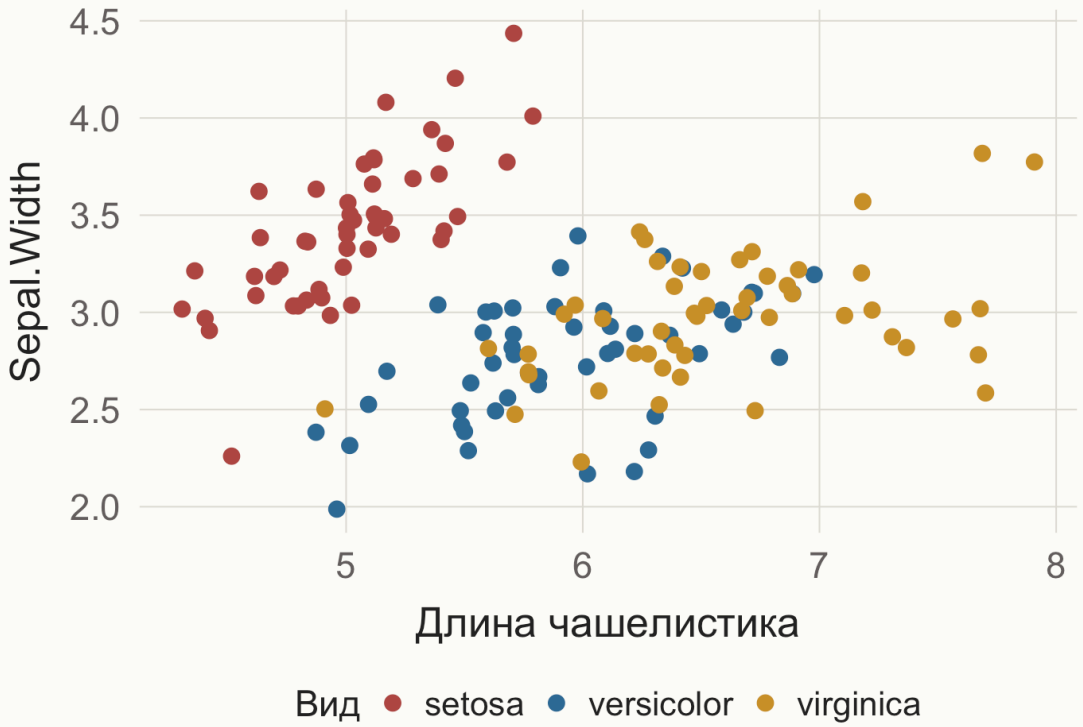
- `scale_x_continuous()`
- `scale_y_*()`
- `scale_color_discrete()`
  - также `scale_colour_*()`
- `scale_fill_*()`
- `scale_shape_*()`
- `scale_linetype_*()`
- `scale_size_*()`

> Категориальные переменные называют также факторами, дискретными или качественными — в зависимости от контекста и собеседника.

scale_*_*()

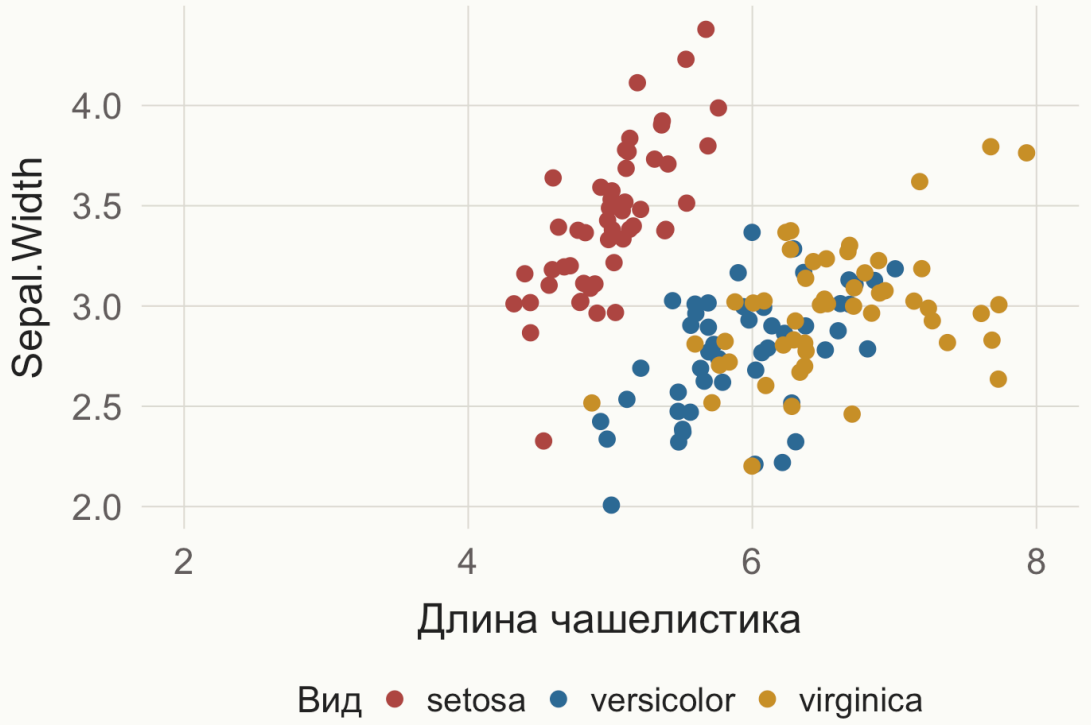
```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_point(position = "jitter") +
  scale_x_continuous("Длина чашелистика") +
  scale_color_discrete("Вид")
```

- У функций шкал много аргументов.
- Первый аргумент — всегда название шкалы (подпись оси или легенды).
- Затем чаще всего используют limits, breaks, expand и labels.



Аргумент `limits`

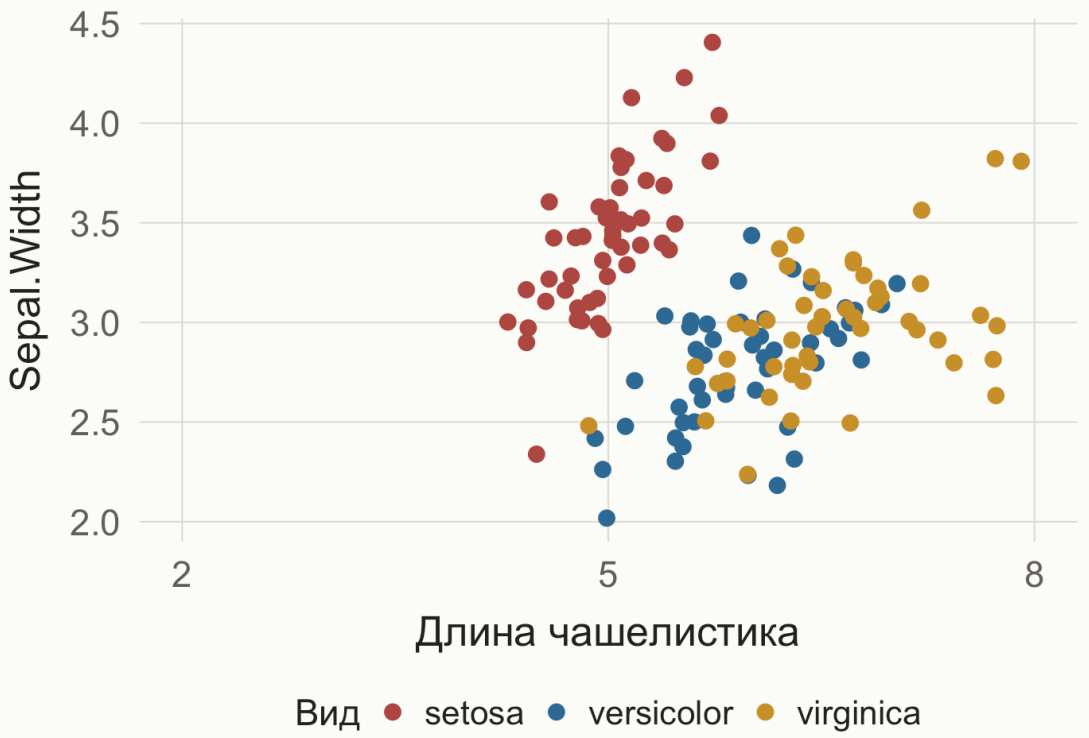
```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_point(position = "jitter") +
  scale_x_continuous("Длина чашелистика",
                     limits = c(2, 8)) +
  scale_color_discrete("Вид")
```



Аргумент `breaks`

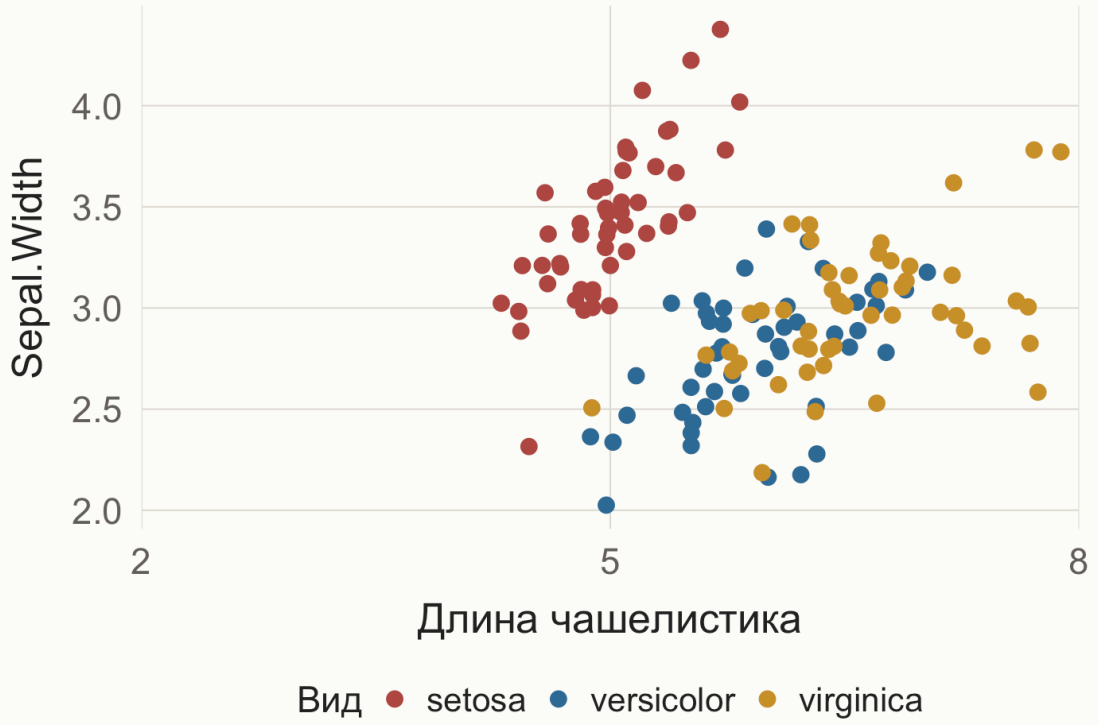
```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_point(position = "jitter") +
  scale_x_continuous("Длина чашелистика",
                     limits = c(2, 8),
                     breaks = seq(2, 8, 3)) +
  scale_color_discrete("Вид")
```

`breaks` управляет положением делений.



Аргумент `expand`

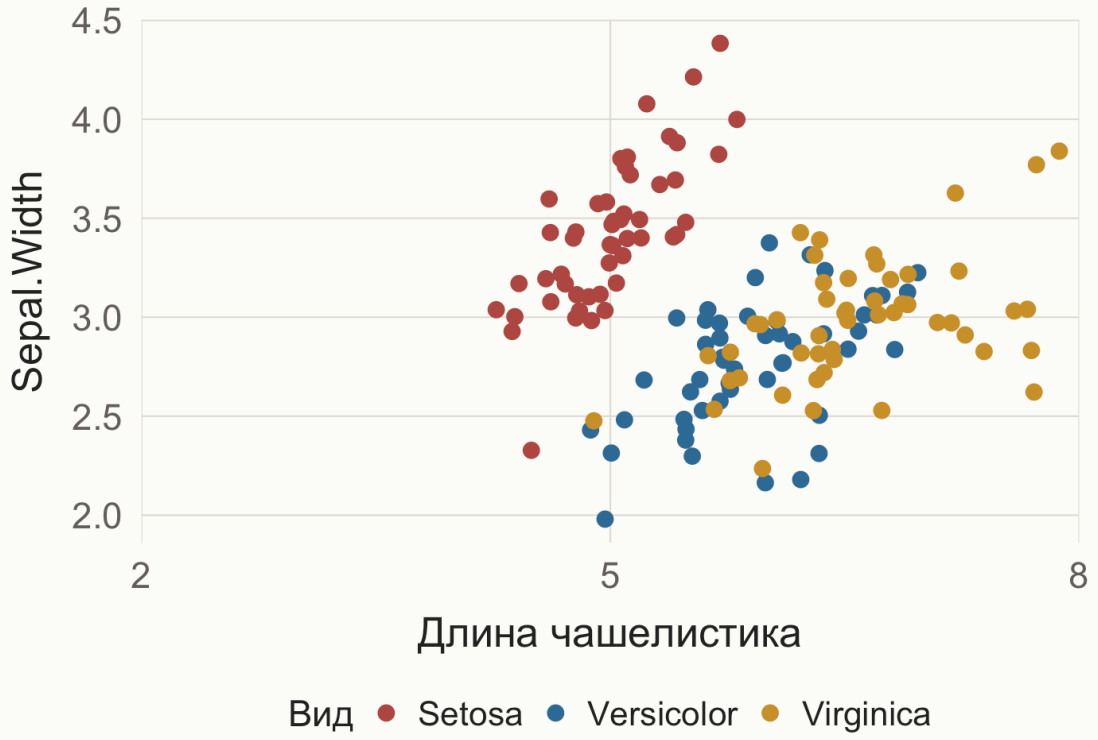
```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_point(position = "jitter") +
  scale_x_continuous("Длина чашелистика",
                     limits = c(2, 8),
                     breaks = seq(2, 8, 3),
                     expand = c(0, 0)) +
  scale_color_discrete("Вид")
```



`expand` добавляет отступ между данными и осями двумя значениями: множителем и фиксированной величиной. `c(0, 0)` убирает отступ совсем.

Аргумент `labels`

```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_point(position = "jitter") +
  scale_x_continuous("Длина чашелистика",
                     limits = c(2, 8),
                     breaks = seq(2, 8, 3),
                     expand = c(0, 0)) +
  scale_color_discrete("Вид",
                       labels = c("Setosa",
                                  "Versicolor",
                                  "Virginica"))
```

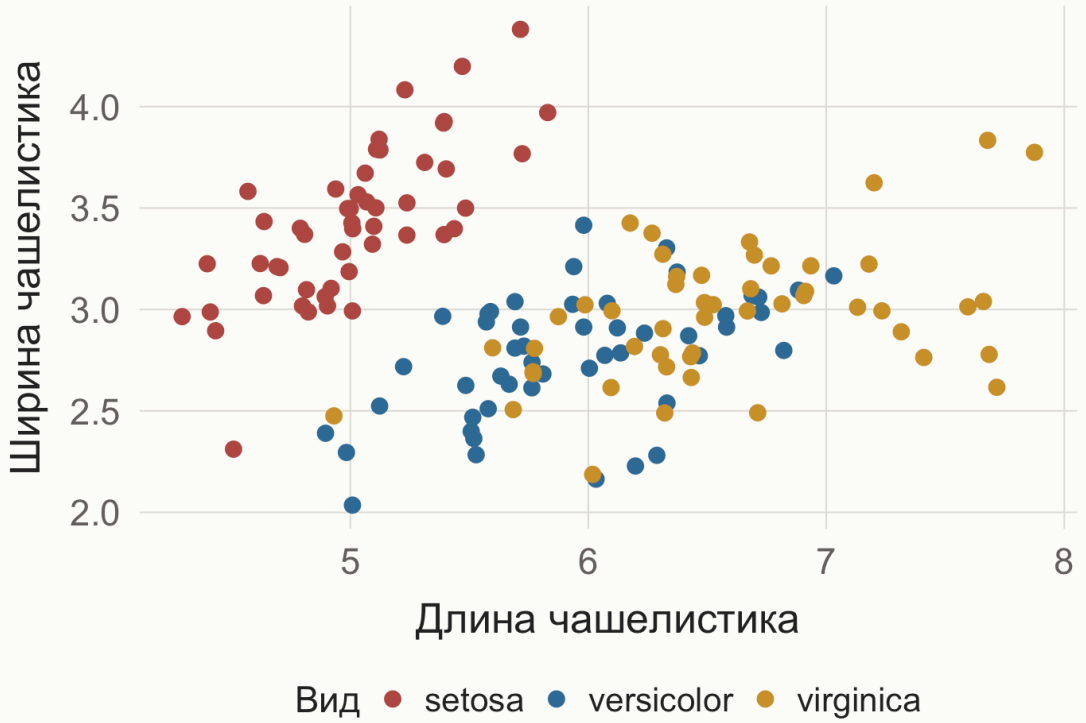


Здесь мы меняем подписи категорий в легенде.

Функция `labs()`

```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_point(position = "jitter") +
  labs(x = "Длина чашелистика",
       y = "Ширина чашелистика",
       color = "Вид")
```

`labs()` позволяет быстро изменить подписи осей и легенды.



Рекомендации по выбору эстетик

Опирайтесь на классические руководства:

- Жак Бертен
  - «Семиология графики» (The Semiology of Graphics), 1967
- Уильям Кливленд
  - The Elements of Graphing Data, 1985
  - Visualizing Data, 1993



Форма следует функции



Форма следует функции

Функция

Основная:

- Точные и эффективные представления.

Второстепенная:

- Привлекательные, красивые графики.

Руководящие принципы

Никогда:

- Искажать или скрывать данные
- Запутывать зрителя сложностью

Всегда:

- Учитывать аудиторию и цель каждого графика.

x	y = f(x)	Группа
5.1	3.5	A
4.9	3.0	A
4.7	3.2	A
4.6	3.1	A
⋮	⋮	⋮
7.0	3.2	B
6.4	3.2	B
6.9	3.1	B
5.5	2.3	B

Трудно получить прямо из данных ↓

Для каждой группы:

n, размах, выбросы

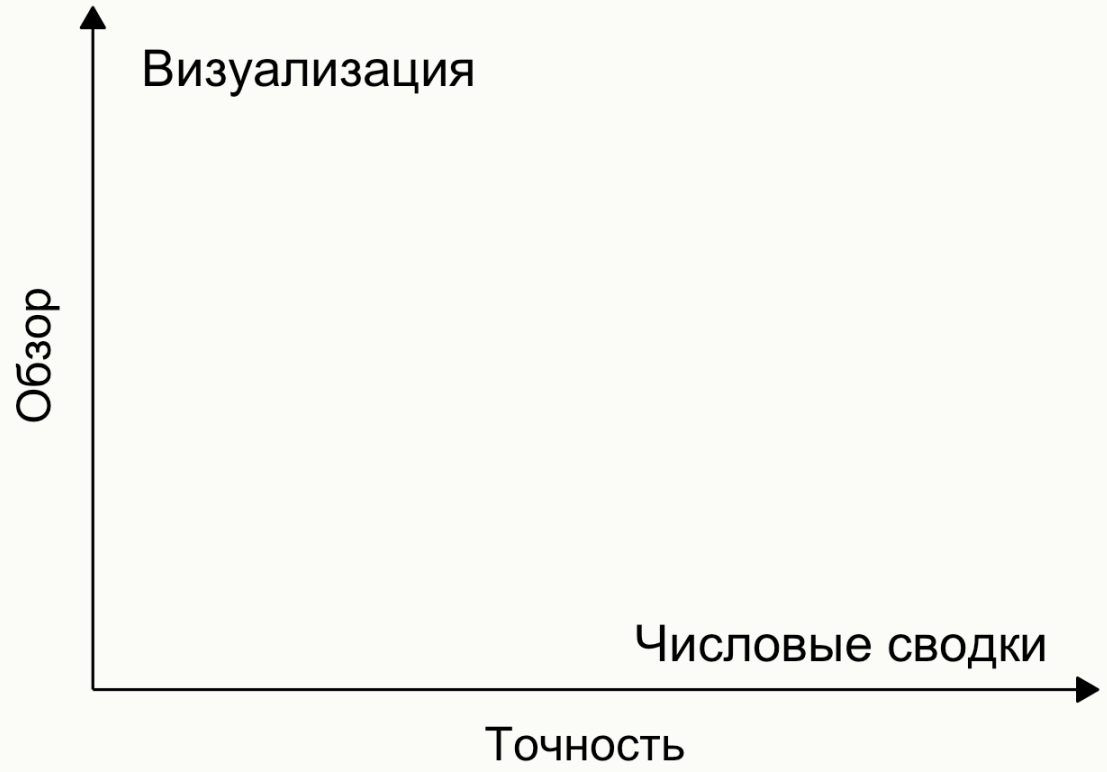
$\bar{x}$ s_x $\bar{y}$ s_y

$y = f(x) = \beta_0 + \beta_1 x$

r R^2

- Две непрерывные переменные, x и y
- y — функция от x
- Есть две группы, A и B
- Получить сводные статистики, просто глядя на данные, довольно трудно

Извлечение информации из данных



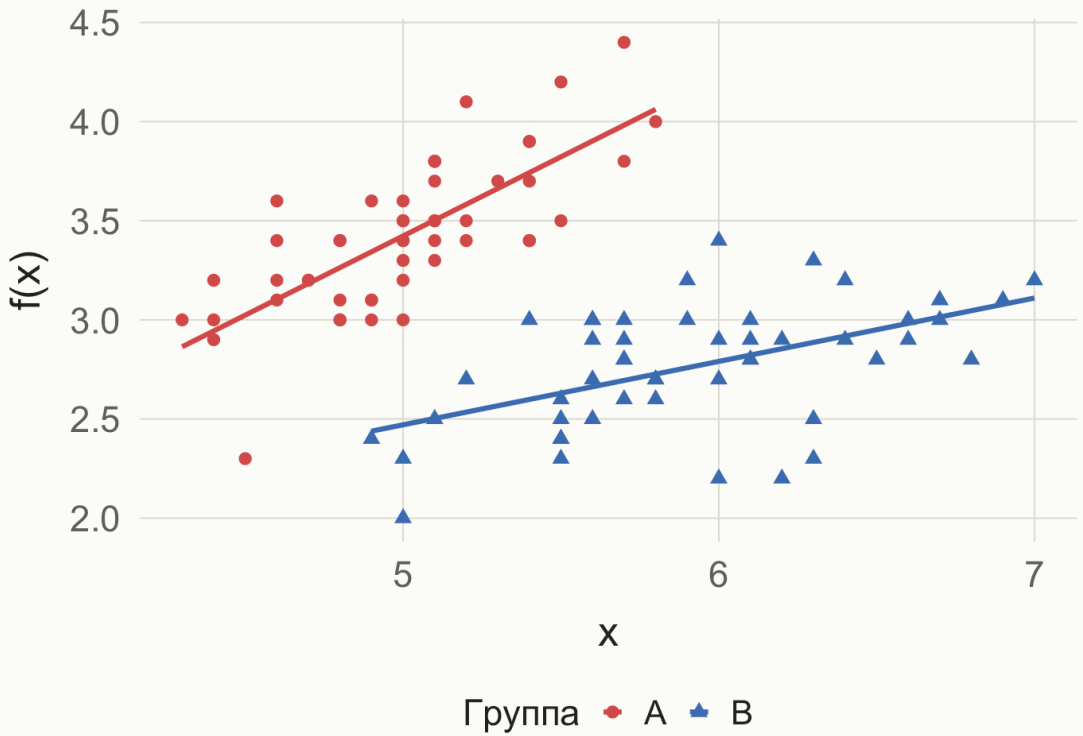
У нас два варианта: числовые сводки — точные, но плохо дают общий обзор, или визуализация — неточная, но отлично даёт общий обзор.

Кодирование чисел в графики

x	y	Группа
5.1	3.5	A
4.9	3.0	A
⋮	⋮	⋮
7.0	3.2	B
6.4	3.2	B

КОДИРОВАНИЕ ⟶
таблица → график

⟵ **ДЕКОДИРОВАНИЕ**
график → n, $\bar{x}$, s_x, β_0, β_1, r, R^2



Кодирование и декодирование по своей природе неточны — примерно как перевод с одного языка на другой.

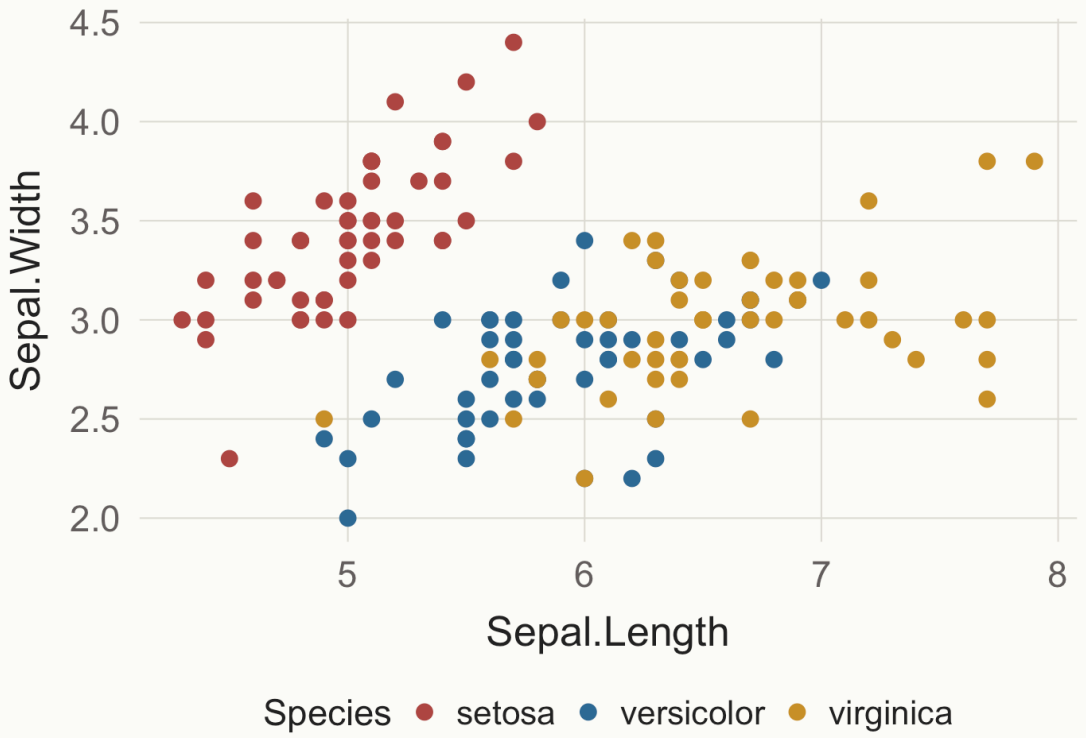
Лучший выбор эстетик

- **Эффективный**
  - Даёт обзор быстрее, чем числовые сводки
- **Точный**
  - Минимизирует потерю информации

Эстетики — непрерывные переменные

```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_point()
```

Диаграмму рассеяния так легко понять, потому что она отображает данные положением на общей шкале.

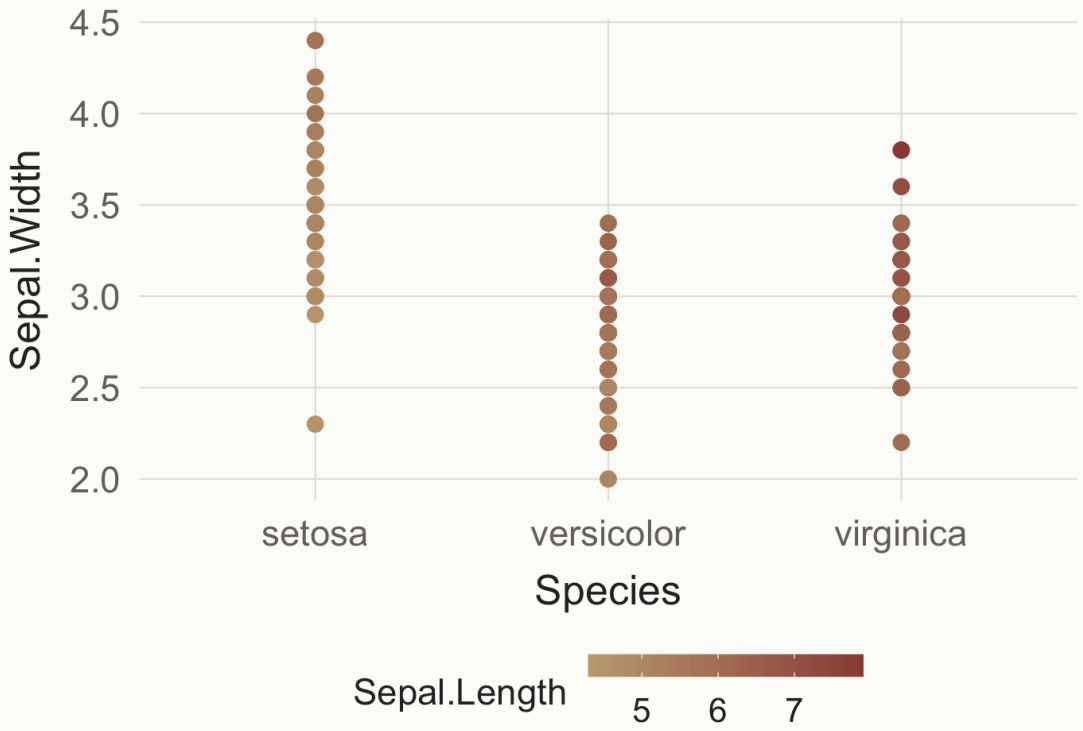


Эстетики — непрерывные переменные

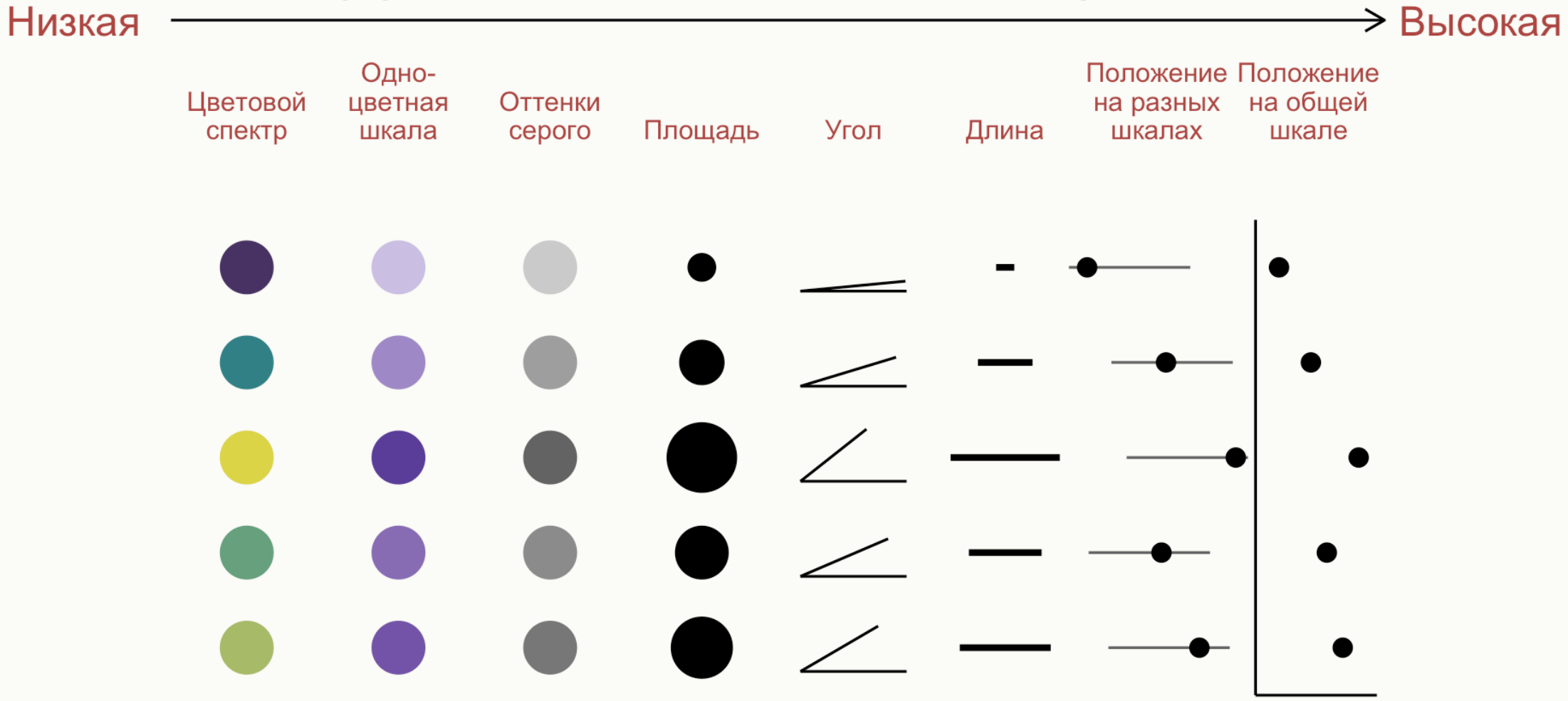
Представьте, что мы поменяли местами отображения x и цвета.

```
ggplot(iris, aes(color = Sepal.Length,
                 y = Sepal.Width,
                 x = Species)) +
  geom_point()
```

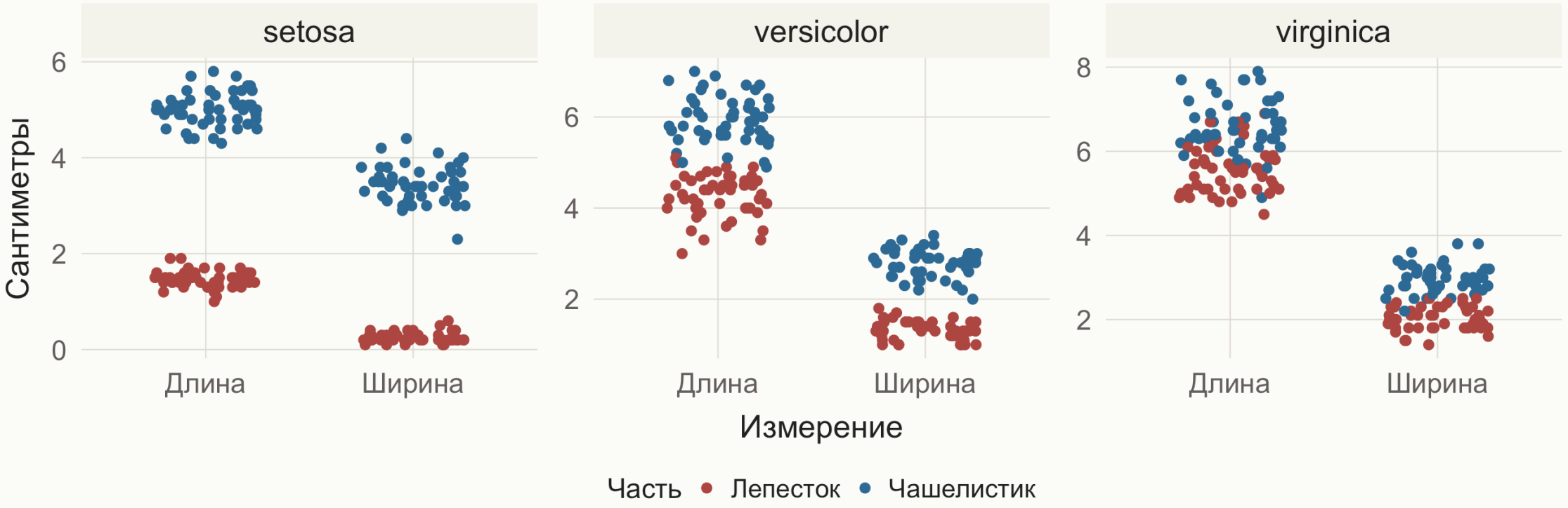
- Ни точно, ни эффективно.
- Невозможно увидеть связь между этими тремя переменными.
- Читатель может понять график не так, как задумано.



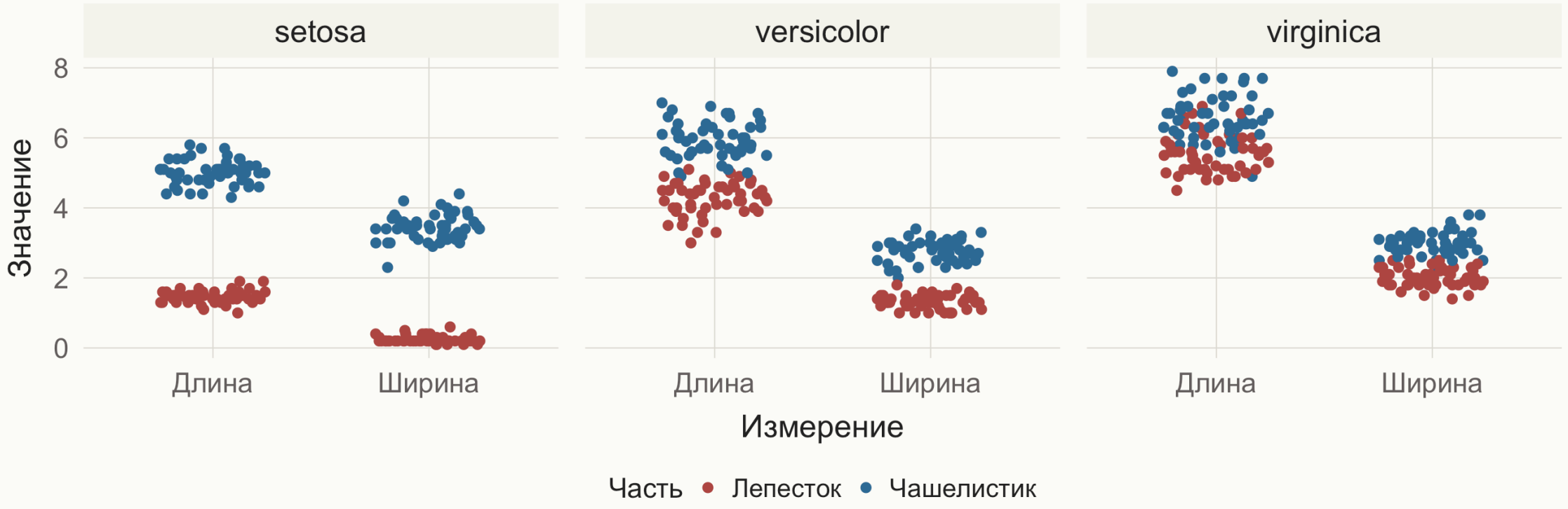
Эффективность и точность декодирования



Три диаграммы рассеяния ирисов, разные оси y



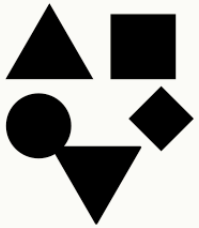
Одна диаграмма с панелями, общая ось y



Эффективность различения групп

Низкая → Высокая

Формы с заливкой



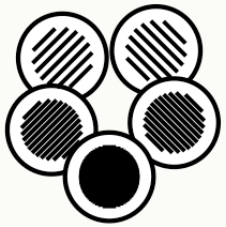
Последовательные цвета



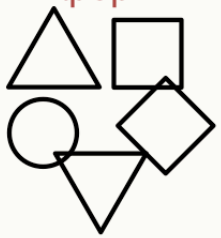
Качественные цвета



Штриховка



Контуры форм



Подписи

ANT1 FRG2

FRG1 Gapdh

DUX4

Толщина линий



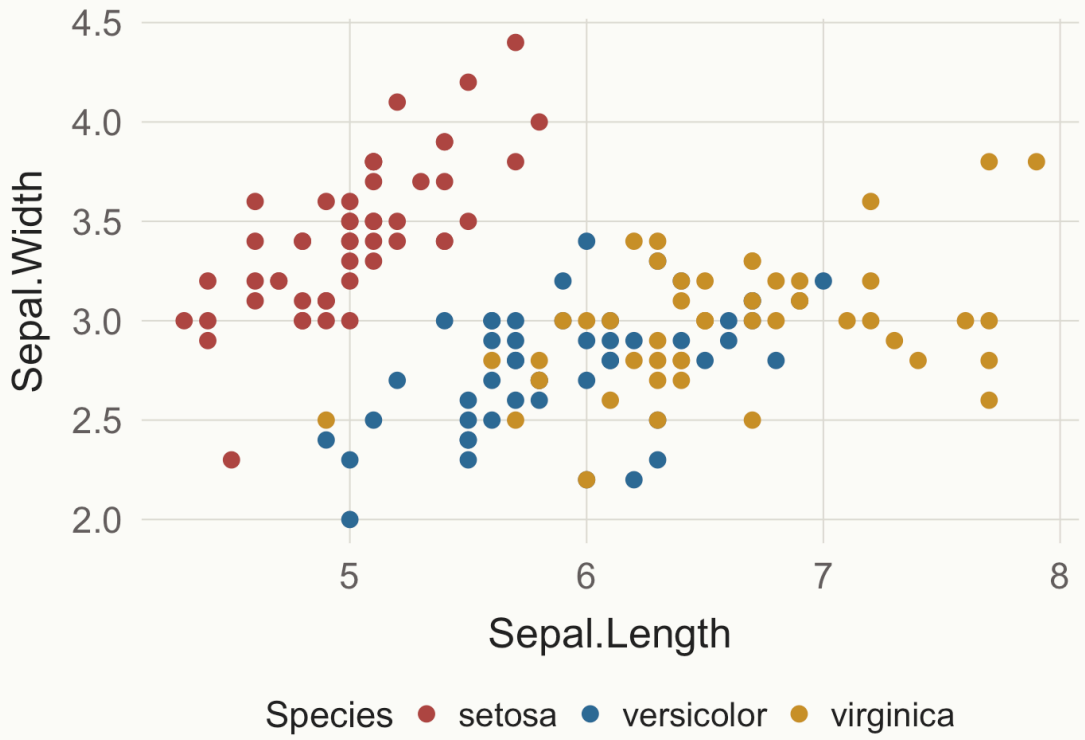
Тип линий



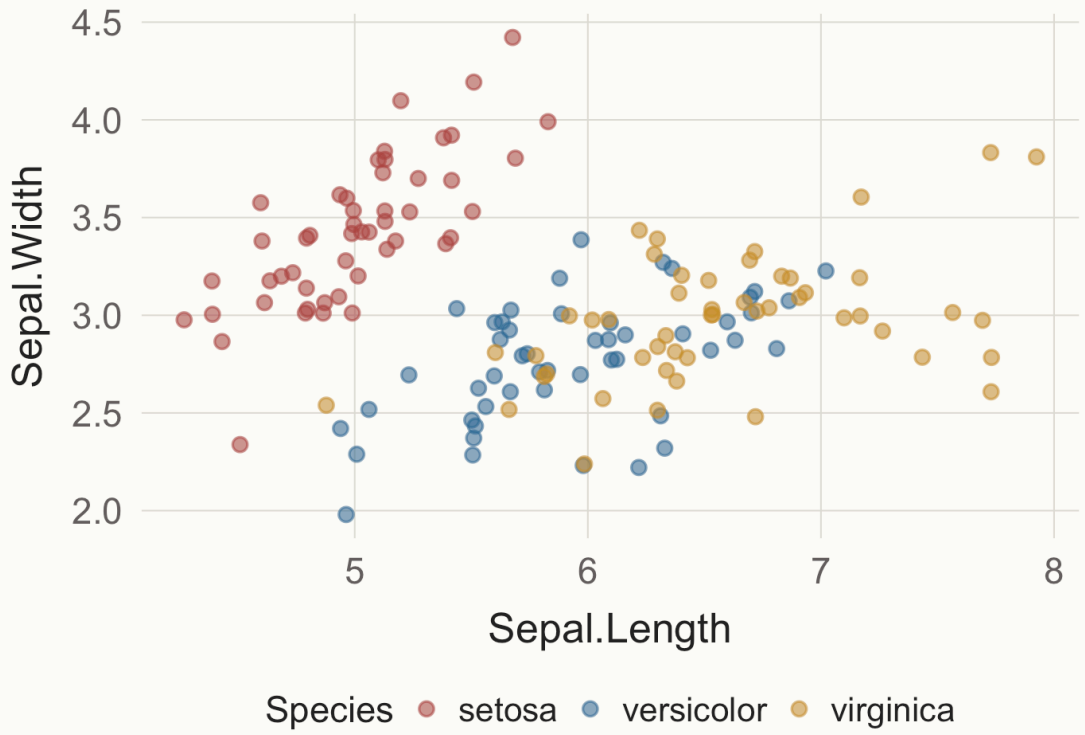
Цвет линий



```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_point()
```



```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_point(position = "jitter",
             alpha = 0.6)
```



Слой геометрии

Определяет, как на самом деле выглядит график.

48 геометрий на выбор

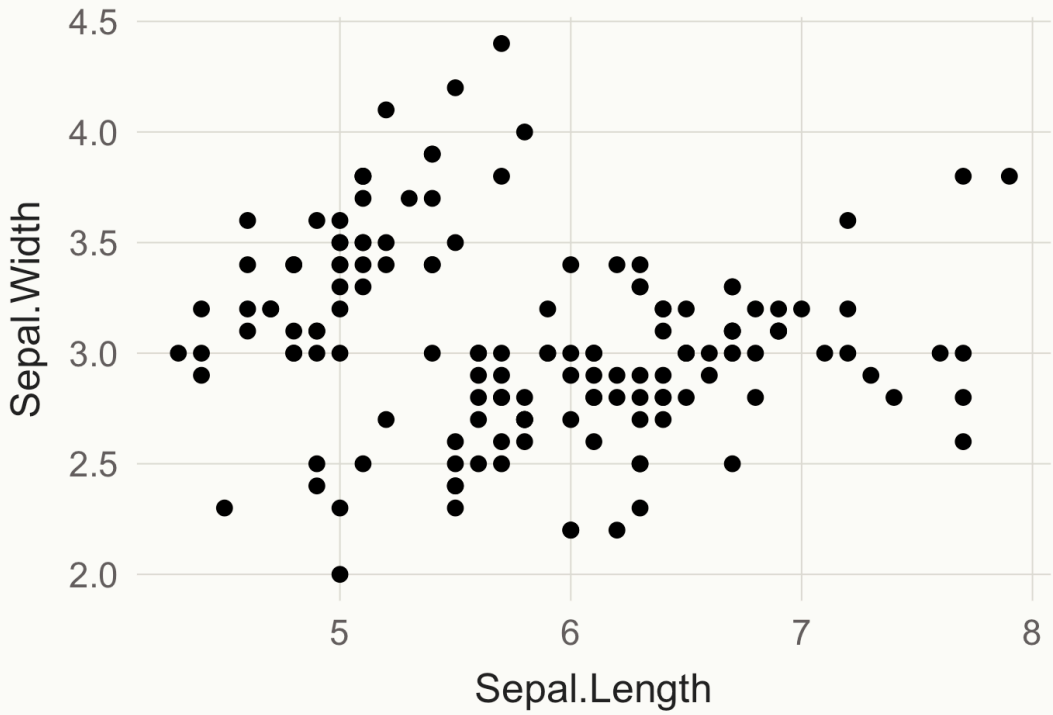
geom_*						
abline	contour	dotplot	jitter	pointrange	ribbon	spoke
area	count	errorbar	label	polygon	rug	step
bar	crossbar	errorbarh	line	qq	segment	text
bin2d	curve	freqpoly	linerange	qq_line	sf	tile
blank	density	hex	map	quantile	sf_label	violin
boxplot	density2d	histogram	path	raster	sf_text	vline
col	density_2d	hline	point	rect	smooth	

Диаграммы рассеяния

Каждая геометрия принимает определённые эстетики, например `geom_point()`:

Обязательные
x, y

```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width)) +
  geom_point()
```

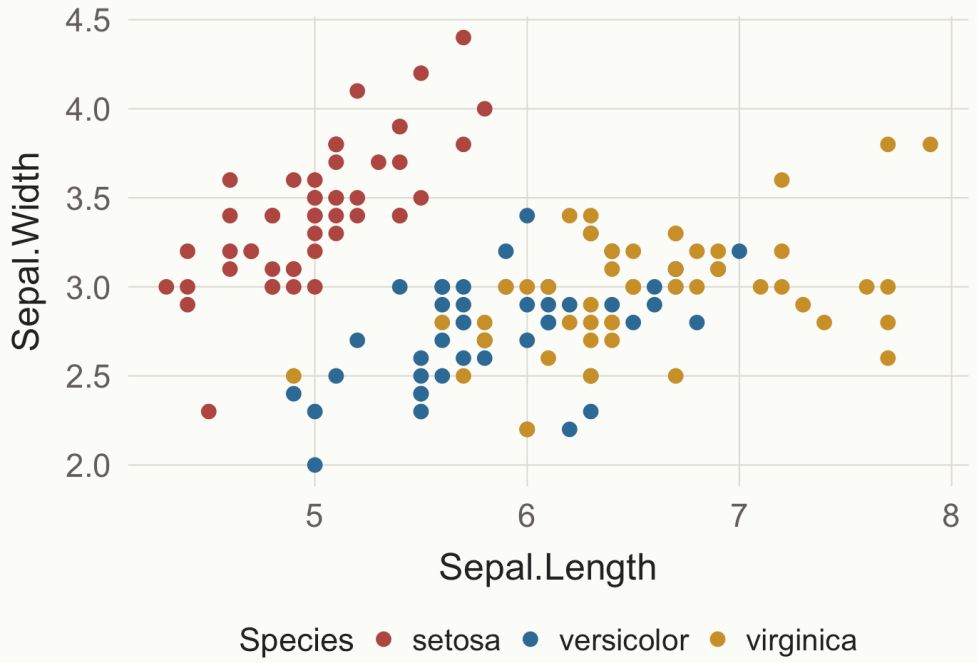


Диаграммы рассеяния

Каждая геометрия принимает определённые эстетики, например `geom_point()`:

Обязательные	Дополнительные
x, y	alpha, color, fill, shape, size, stroke

```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_point()
```

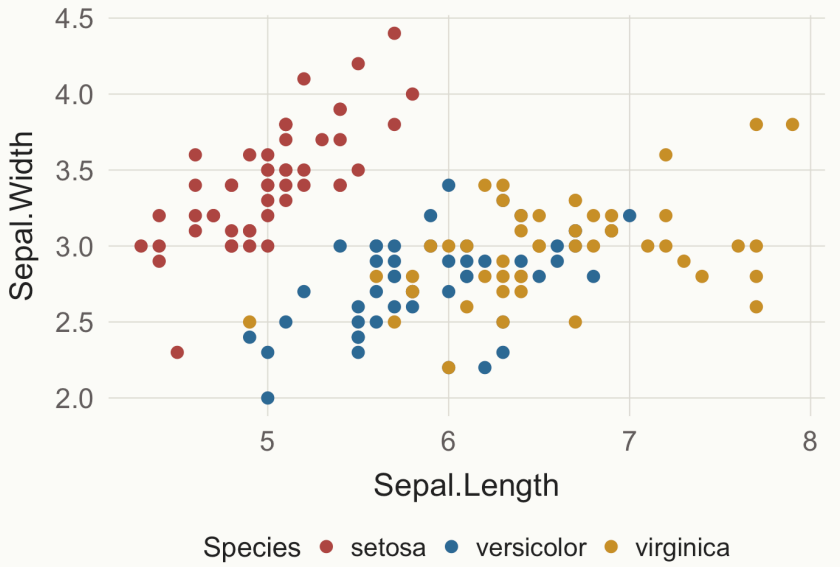


Эстетики отдельной геометрии

```
# Эти варианты дают одинаковый график
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, col = Species)) +
  geom_point()

ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(aes(col = Species))
```

Отображения каждого слоя можно задавать независимо:



```
head(iris, 3) # Исходные данные
```

```
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
```

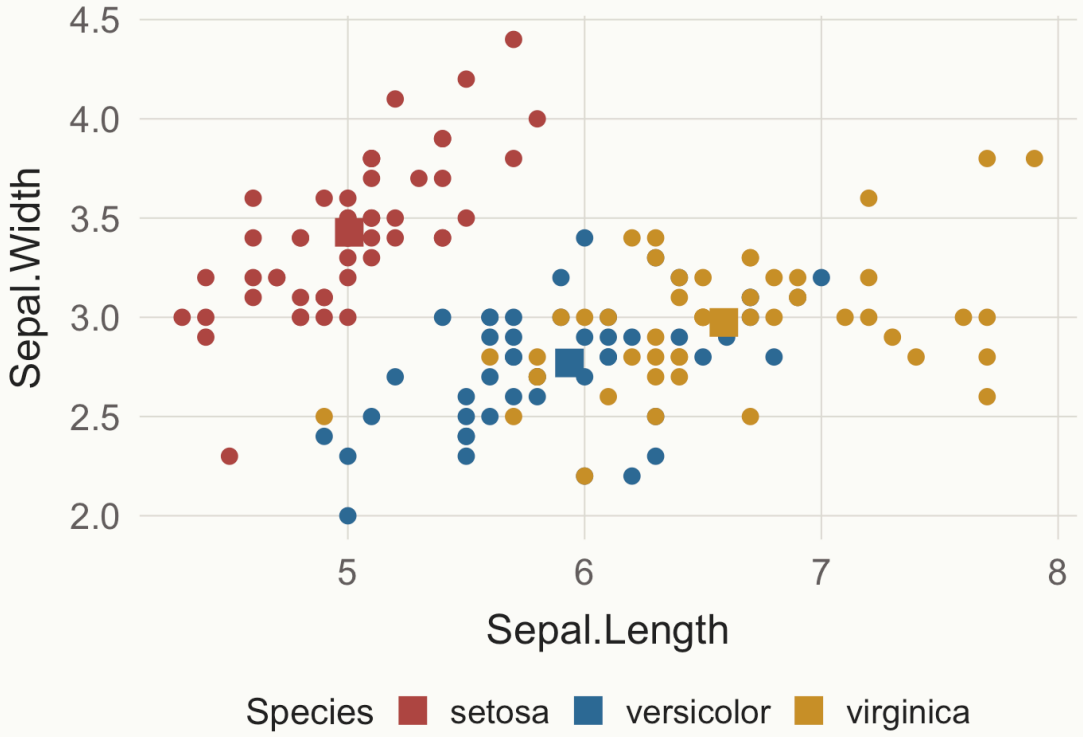
```
library(dplyr)

iris %>%
group_by(Species) %>%
summarise_all(mean) -> iris.summary
```

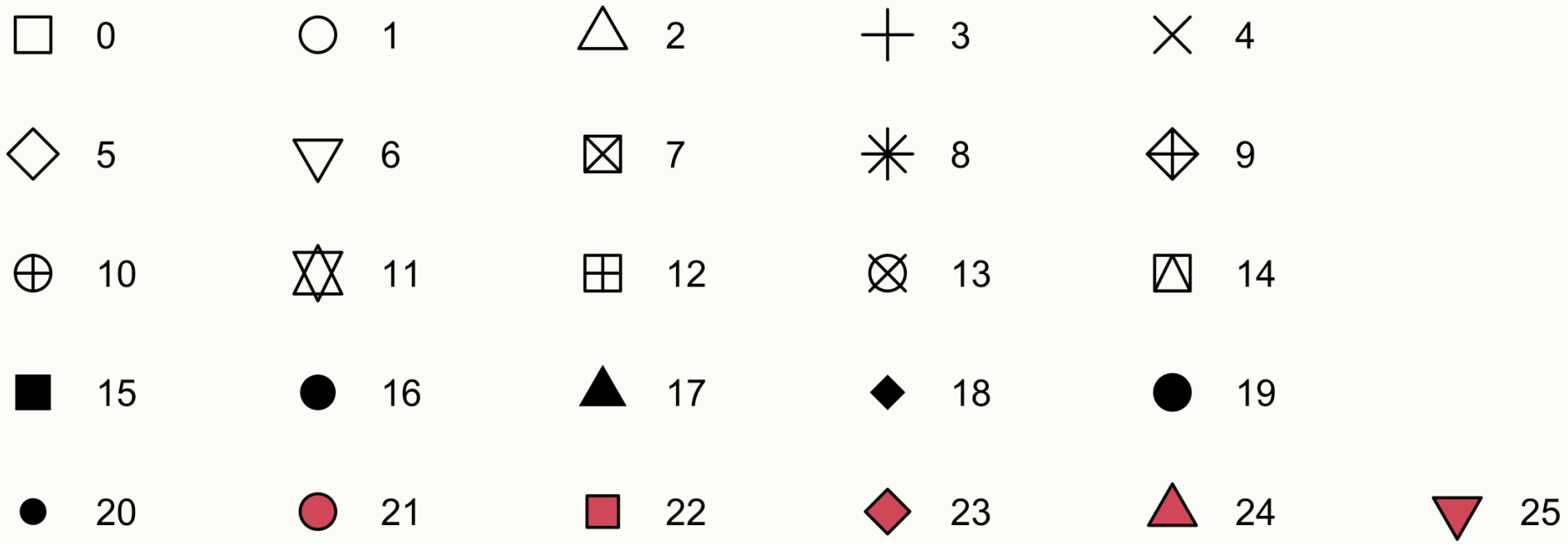
```
# A tibble: 3 × 5
  Species    Sepal.Length Sepal.Width Petal.Length Petal.Width
  <fct>             <dbl>       <dbl>        <dbl>       <dbl>
1 setosa             5.01        3.43         1.46       0.246
2 versicolor         5.94        2.77         4.26       1.33
3 virginica          6.59        2.97         5.55       2.03
```

Чтобы показать все наблюдения и поверх них — средние x и y:

```
ggplot(iris, aes(
    x = Sepal.Length,
    y = Sepal.Width,
    col = Species)) +
  # Наследует и данные, и aes из ggplot()
  geom_point() +
  # Другие данные, но унаследованные aes
  geom_point(data = iris.summary,
             shape = 15,
             size = 5)
```

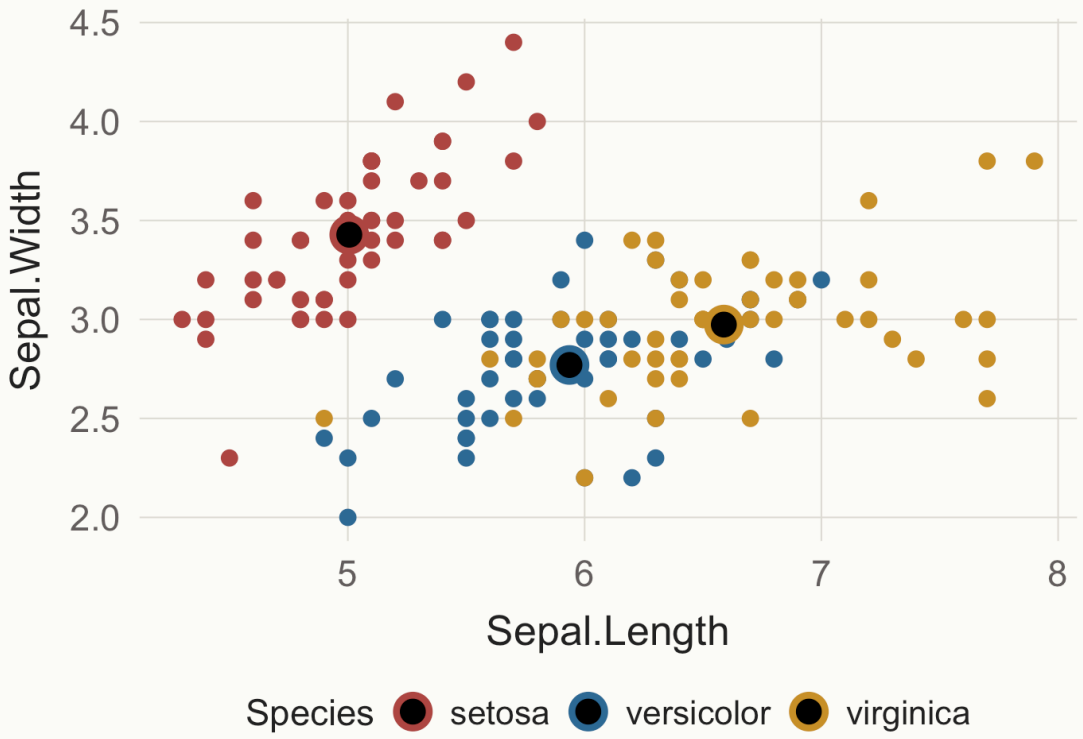


Значения атрибута shape



Пример

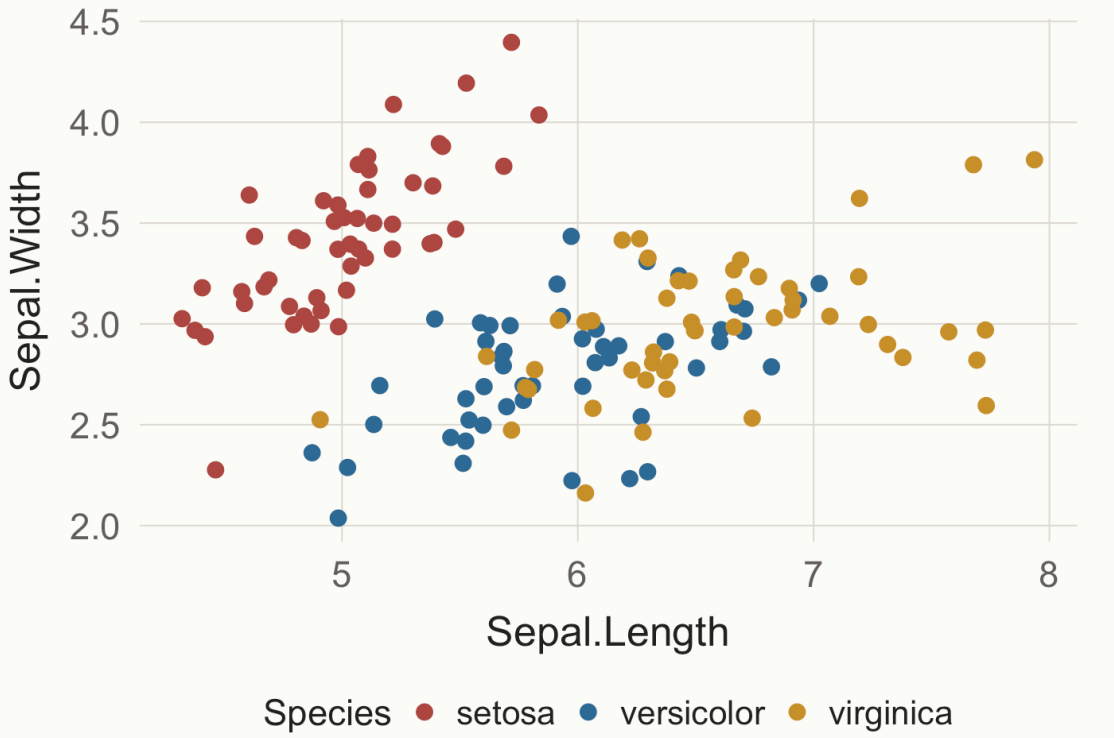
```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 col = Species)) +
  geom_point() +
  geom_point(data = iris.summary,
             shape = 21,
             size = 5,
             fill = "black",
             stroke = 2)
```



`geom_jitter()`

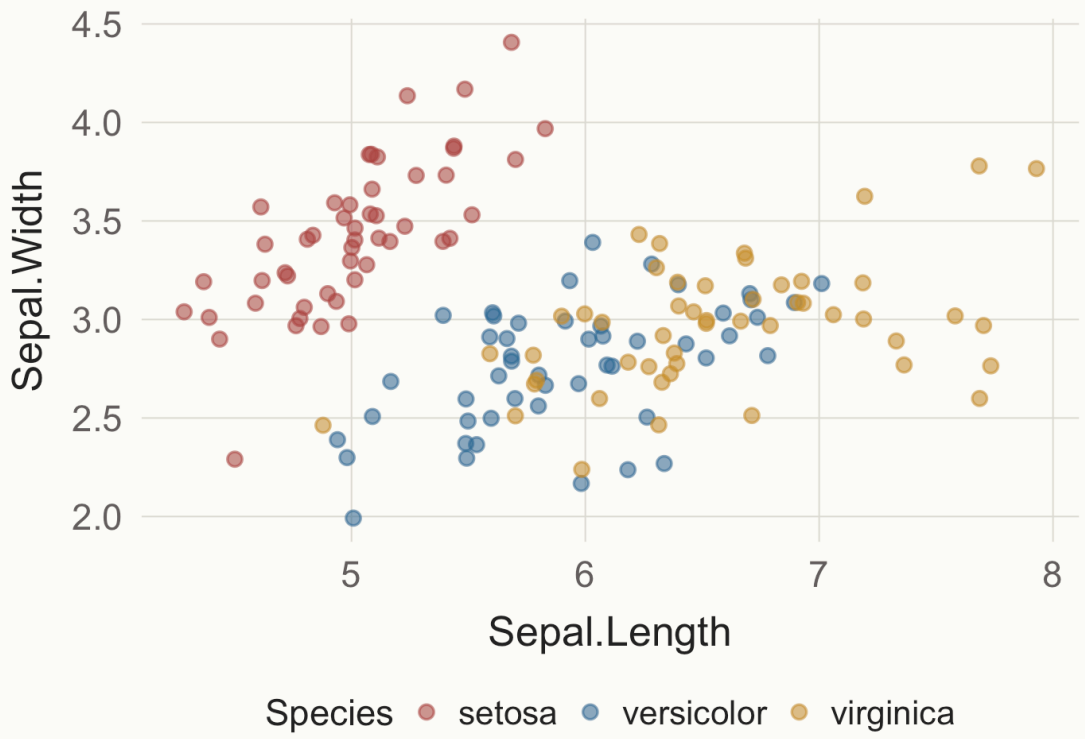
Сокращение для `geom_point(position = "jitter")`

```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_jitter()
```



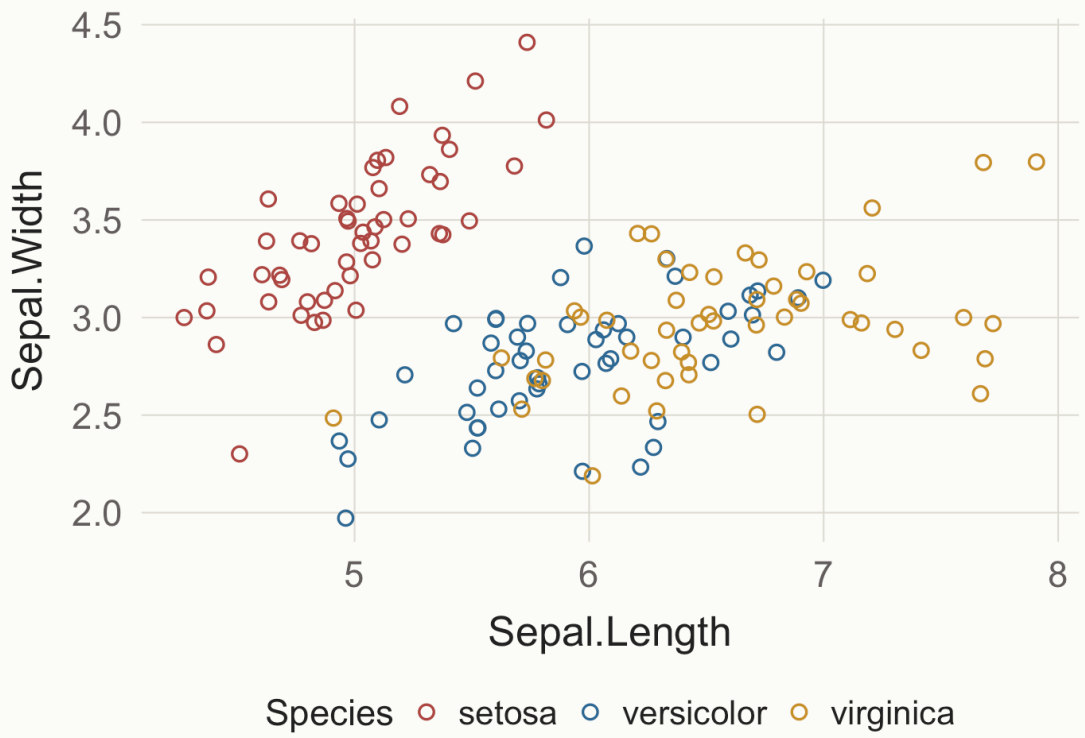
Не забудьте настроить alpha

```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_jitter(alpha = 0.6)
```



Полые круги тоже помогают

```
ggplot(iris, aes(x = Sepal.Length,
                 y = Sepal.Width,
                 color = Species)) +
  geom_jitter(shape = 1)
```



Столбцы, линии и темы

Гистограммы

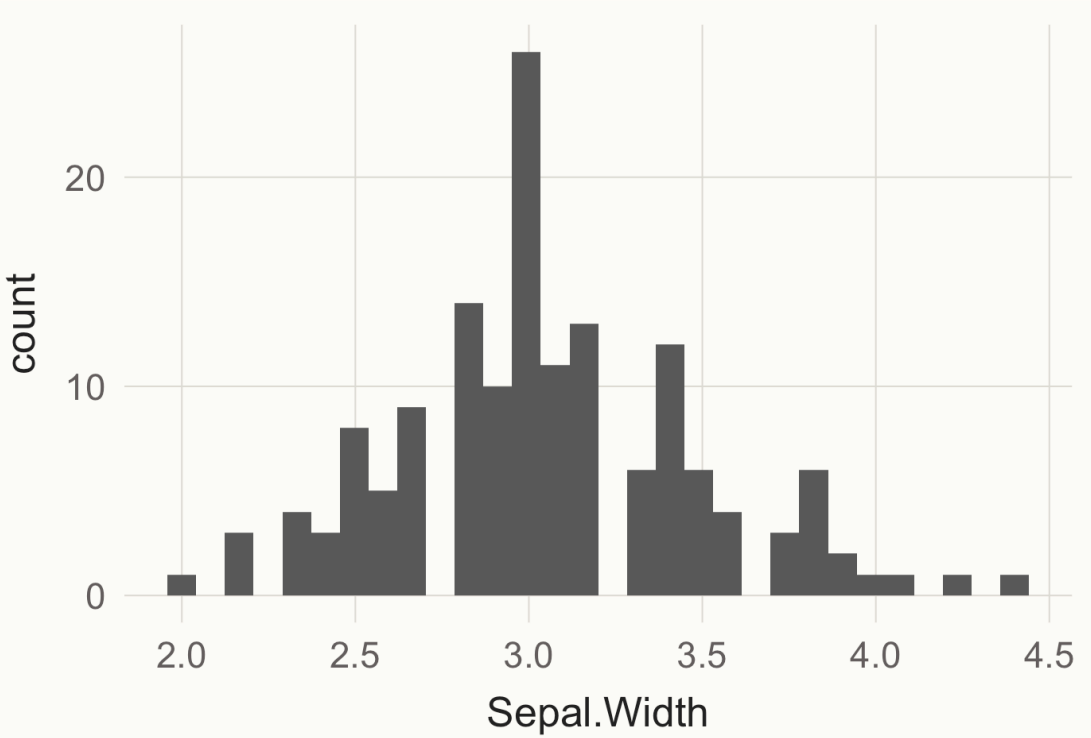
Гистограмма — распределение непрерывной переменной, разбитой на интервалы.

```
ggplot(iris, aes(x = Sepal.Width)) +
    geom_histogram()
```

Здесь нужна только одна эстетика `x`.

```
# Ширина интервала по умолчанию
diff(range(iris$Sepal.Width))/30
```

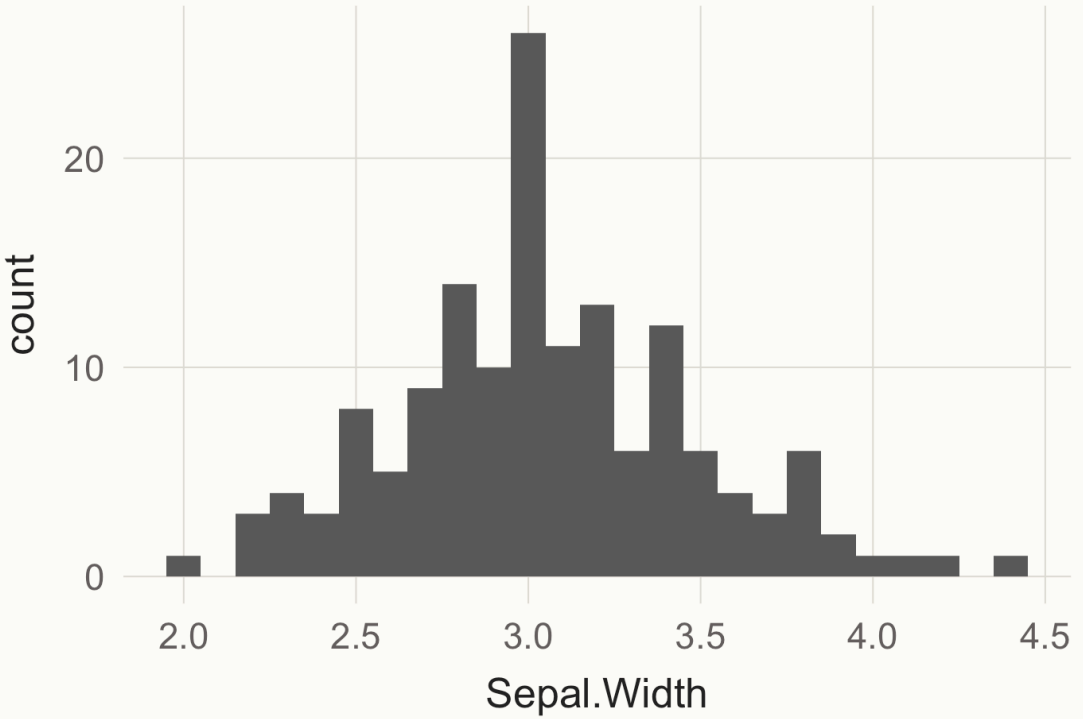
```
[1] 0.08
```



Понятная и осмысленная ширина интервала

```
ggplot(iris, aes(x = Sepal.Width)) +
    geom_histogram(binwidth = 0.1)
```

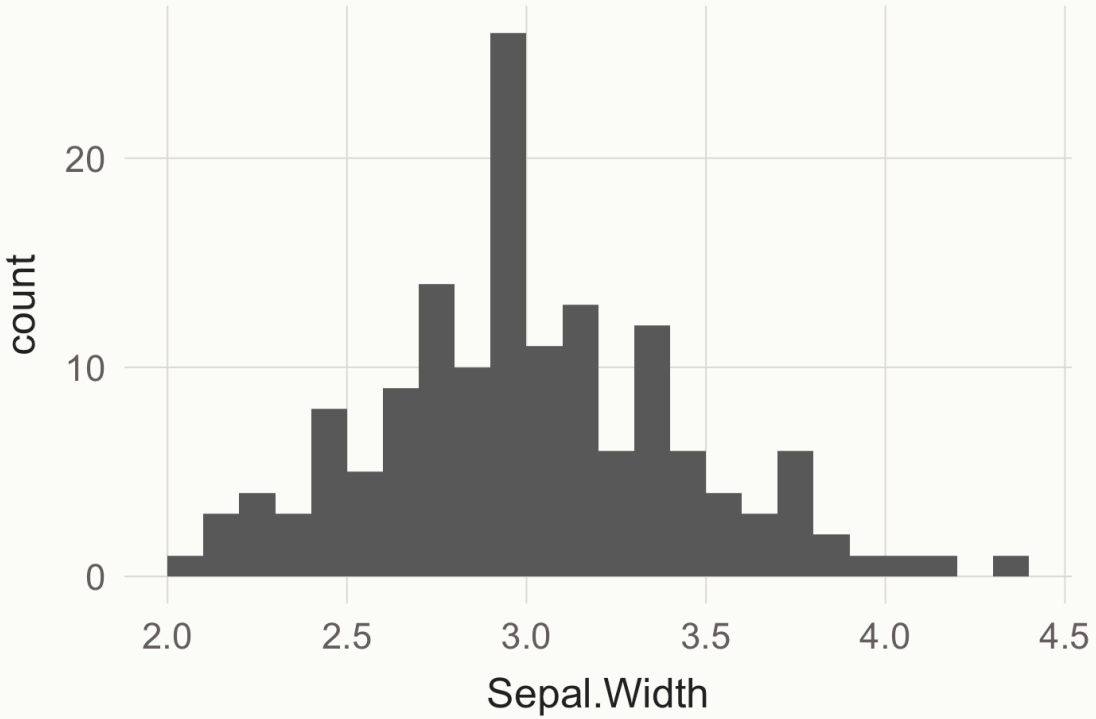
- Всегда задавайте ширину интервала, осмысленную для ваших данных.
- Между столбиками нет промежутков.
- Гистограмма представляет непрерывное распределение.



Перенос делений

```
ggplot(iris, aes(x = Sepal.Width)) +
    geom_histogram(binwidth = 0.1,
                   center = 0.05)
```

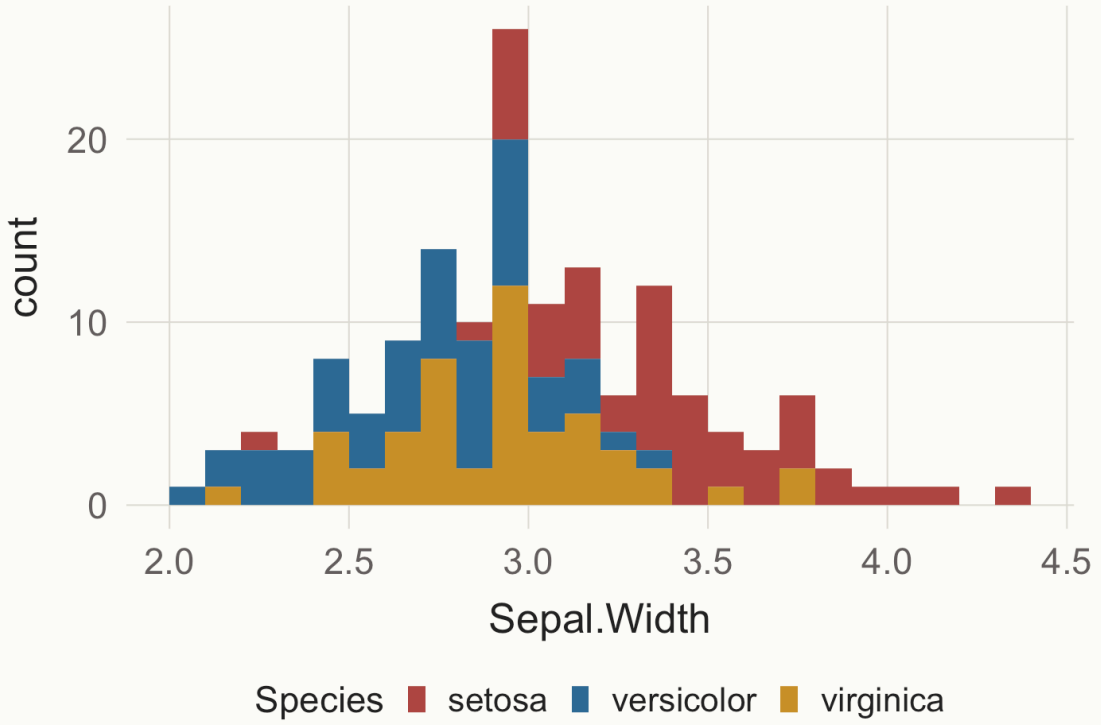
- `center = 0.05` сдвигает интервалы: их границы — 2.0, 2.1, 2.2, ...
- Деления оси X должны совпадать с границами столбиков.
- Мы изображаем интервалы, а не отдельные значения.



Разные виды

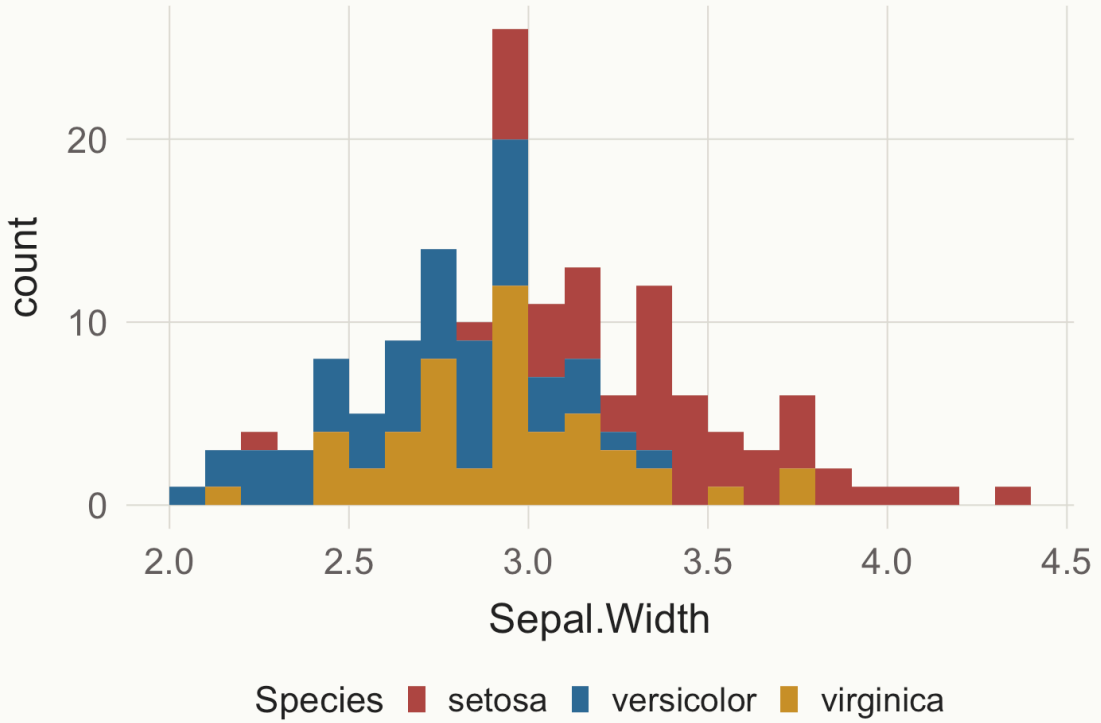
```
ggplot(iris, aes(x = Sepal.Width,
                 fill = Species)) +
    geom_histogram(binwidth = 0.1,
                   center = 0.05)
```

- Столбики можно раскрасить по видам.
- Проблема восприятия: не сразу понятно, перекрываются столбики или стоят друг на друге.



Позиция по умолчанию — "stack"

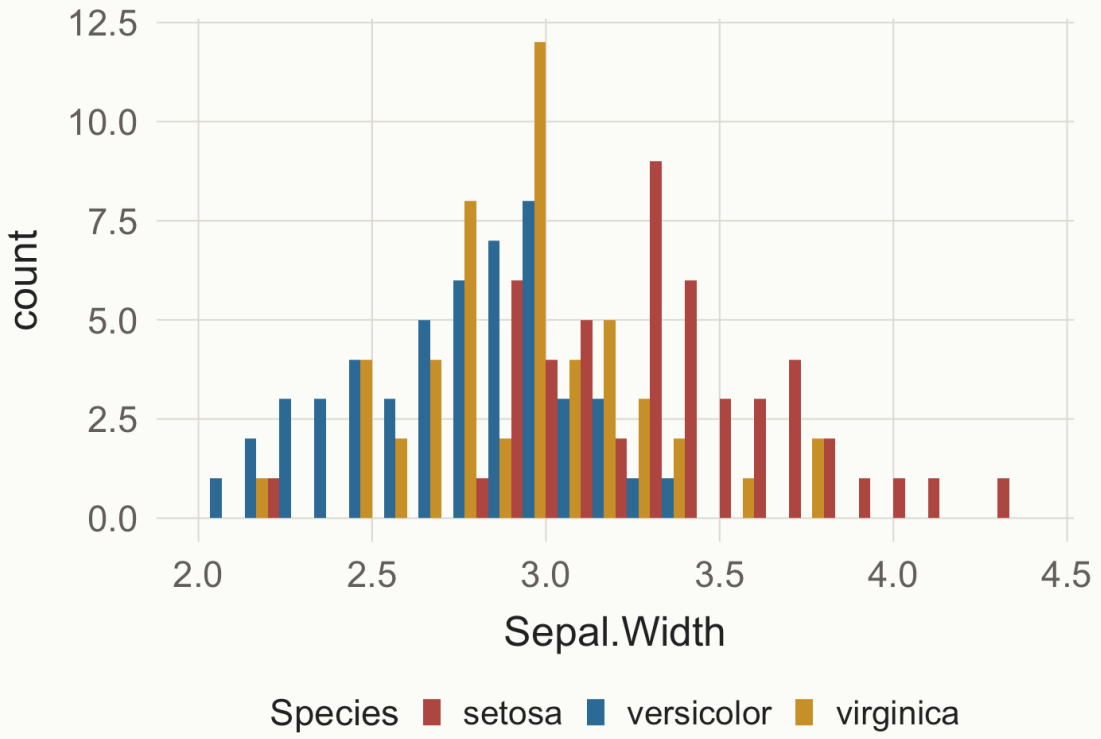
```
ggplot(iris, aes(x = Sepal.Width,
                 fill = Species)) +
    geom_histogram(binwidth = 0.1,
                   center = 0.05,
                   position = "stack")
```



position = "dodge"

```
ggplot(iris, aes(x = Sepal.Width,
                 fill = Species)) +
    geom_histogram(binwidth = 0.1,
                   center = 0.05,
                   position = "dodge")
```

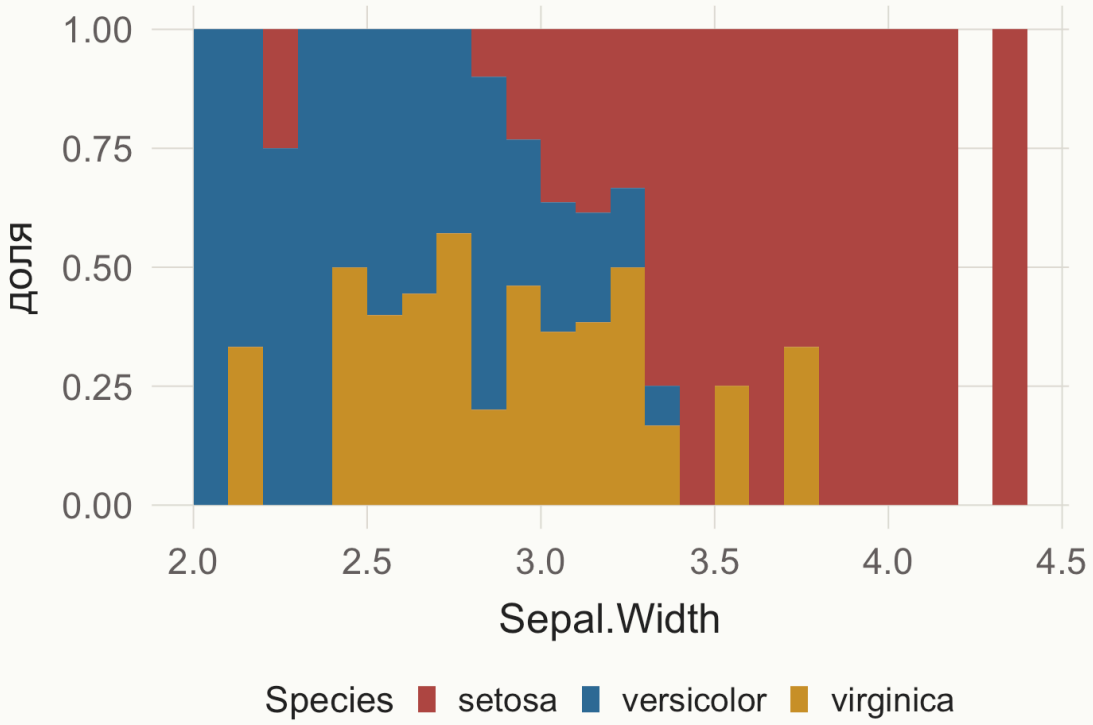
- Столбики разных видов ставятся рядом друг с другом.
- Из-за множества узких столбиков трудно понять, что происходит.



position = "fill"

```
ggplot(iris, aes(x = Sepal.Width,
                 fill = Species)) +
    geom_histogram(binwidth = 0.1,
                   center = 0.05,
                   position = "fill") +
    labs(y = "доля")
```

- Нормирует каждый интервал: столбик показывает, какую долю наблюдений в нём составляет каждый вид.
- Меняем подпись оси y на «доля» вместо «count».



Столбчатые диаграммы

- Гистограмму можно считать частным случаем столбчатой диаграммы, где непрерывная ось X разбита на интервалы.
- Классическая столбчатая диаграмма имеет категориальную ось X.
- Здесь нужна `geom_bar` или `geom_col`.

Геометрия	Stat	Действие
geom_bar()	"count"	Считает число наблюдений в каждой позиции x
geom_col()	"identity"	Изображает сами значения

Привычки млекопитающих

```
sleep <- msleep[!is.na(msleep$vore),
                c("vore", "sleep_total", "sleep_rem")]
str(sleep)
```

```
tibble [76 × 3] (S3: tbl_df/tbl/data.frame)
 $ vore       : chr [1:76] "carni" "omni" "herbi" "omni" ...
 $ sleep_total: num [1:76] 12.1 17 14.4 14.9 4 14.4 8.7 10.1 3 5.3 ...
 $ sleep_rem  : num [1:76] NA 1.8 2.4 2.3 0.7 2.2 1.4 2.9 NA 0.6 ...
```

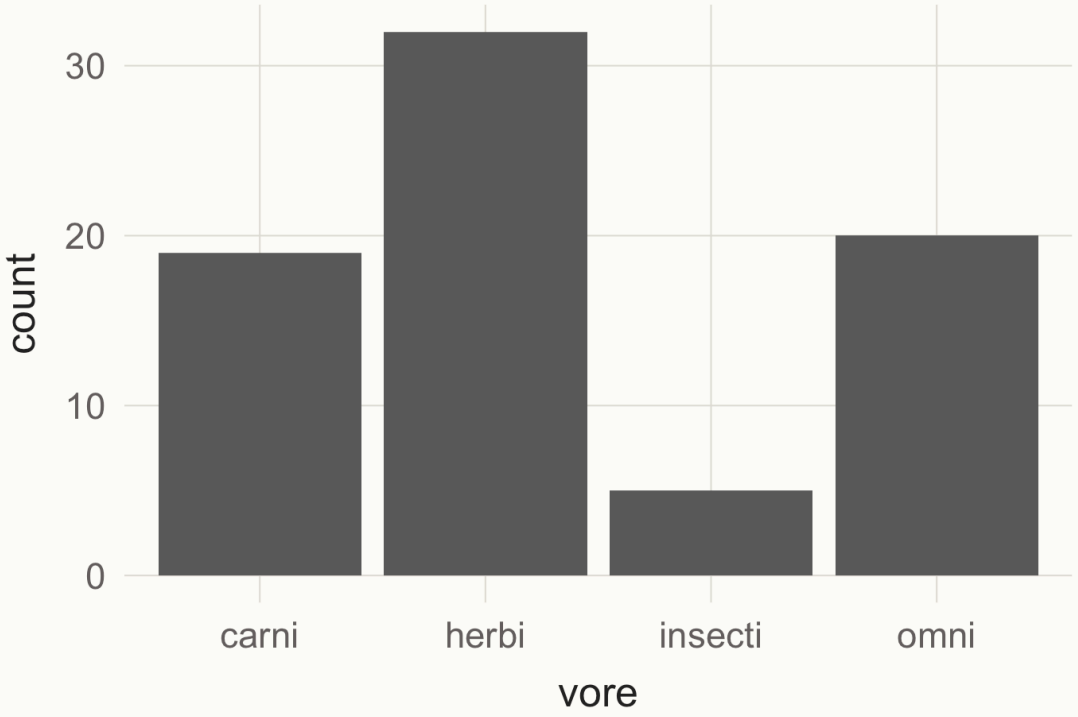
Набор данных о продолжительности сна (всего и в фазе REM) и типе питания разных млекопитающих.

vore — тип питания: carni — хищные, herbi — растительноядные, insecti — насекомоядные, omni — всеядные.

Столбчатая диаграмма

```
ggplot(sleep, aes(vore)) +
    geom_bar()
```

- Мы разделяем данные по типу питания.
- Мы считаем, сколько наблюдений в каждой категории.



Распределения вместо абсолютных частот

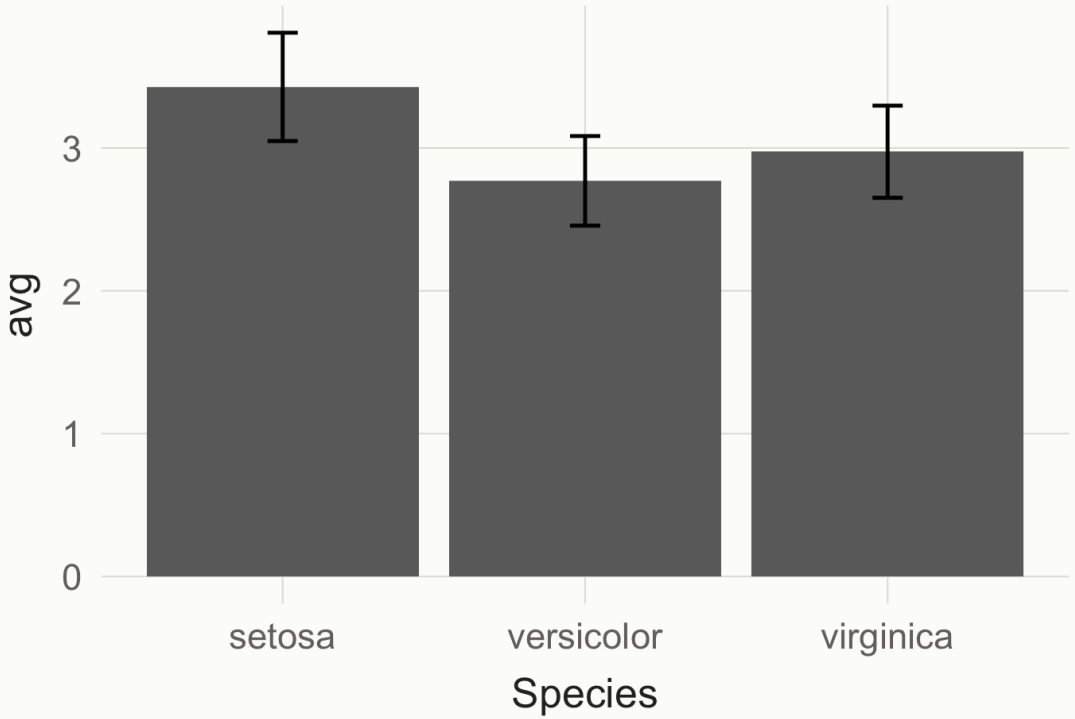
```
library(dplyr)
library(tidyr)
# Описательные статистики:
iris_summ_long <- iris %>%
 select(Species, Sepal.Width) %>%
 pivot_longer(!Species, names_to = "key",
              values_to = "value") %>%
 group_by(Species) %>%
 summarise(avg = mean(value),
           stdev = sd(value))
```

```
iris_summ_long
```

```
# A tibble: 3 × 3
  Species      avg stdev
  <fct>      <dbl> <dbl>
1 setosa      3.43 0.379
2 versicolor  2.77 0.314
3 virginica   2.97 0.322
```

Статистики распределения на графике

```
ggplot(iris_summ_long,
       aes(x = Species, y = avg)) +
 geom_col() +
 geom_errorbar(aes(ymin = avg - stdev,
                   ymax = avg + stdev),
               width = 0.1)
```



- Такие графики называют «динамитными»
- Они часто встречаются в публикациях
- Их не считают удачными: столбик и «усы» скрывают распределение и число наблюдений.

Линейные графики

Особенно полезны для временных рядов.

```
library(lubridate)
library(dplyr)

# В beaver2 время — число вида 930 (09:30), day — номер дня в году.
beaver <- beaver2 %>%
  mutate(
    timestamp = as.POSIXct(paste("1990-11-03",
                                 format(strptime(sprintf("%04d", time), "%H%M"), "%H:%M")),
                           format="%Y-%m-%d %H:%M") + days(day - min(day))
  ) %>%
  select(-day, -time)

str(beaver)
```

```
'data.frame': 100 obs. of  3 variables:
 $ temp     : num  36.6 36.7 36.9 37.1 37.2 ...
 $ activ    : num  0 0 0 0 0 0 0 0 0 0 ...
 $ timestamp: POSIXct, format: "1990-11-03 09:30:00" "1990-11-03 09:40:00" ...
```

Временной ряд для бобра

```
ggplot(beaver, aes(timestamp, temp)) +
  geom_line()
```

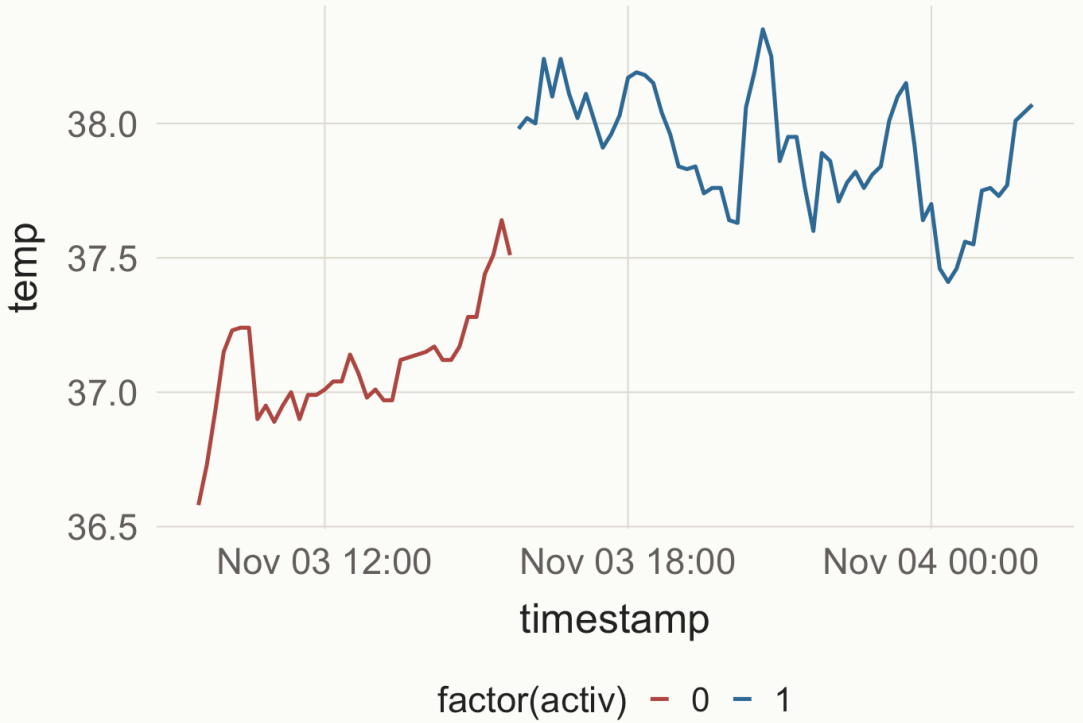
- Наши данные — измерения температуры бобра с интервалом 10 минут.
- Переменная времени имеет формат даты-времени, с которым ggplot2 хорошо работает.



Бобр в цвете

```
ggplot(beaver, aes(timestamp, temp,
                   color = factor(activ))) +
  geom_line()
```

- `activ`: 0 — бобр в укрытии (хатке), 1 — активен снаружи.
- `factor()` делает 0 и 1 категориями, поэтому цвета дискретные.



Набор данных об улове рыб

```
fish = load("data/fish.RData")  # создаёт fish.tidy
str(fish.tidy)
```

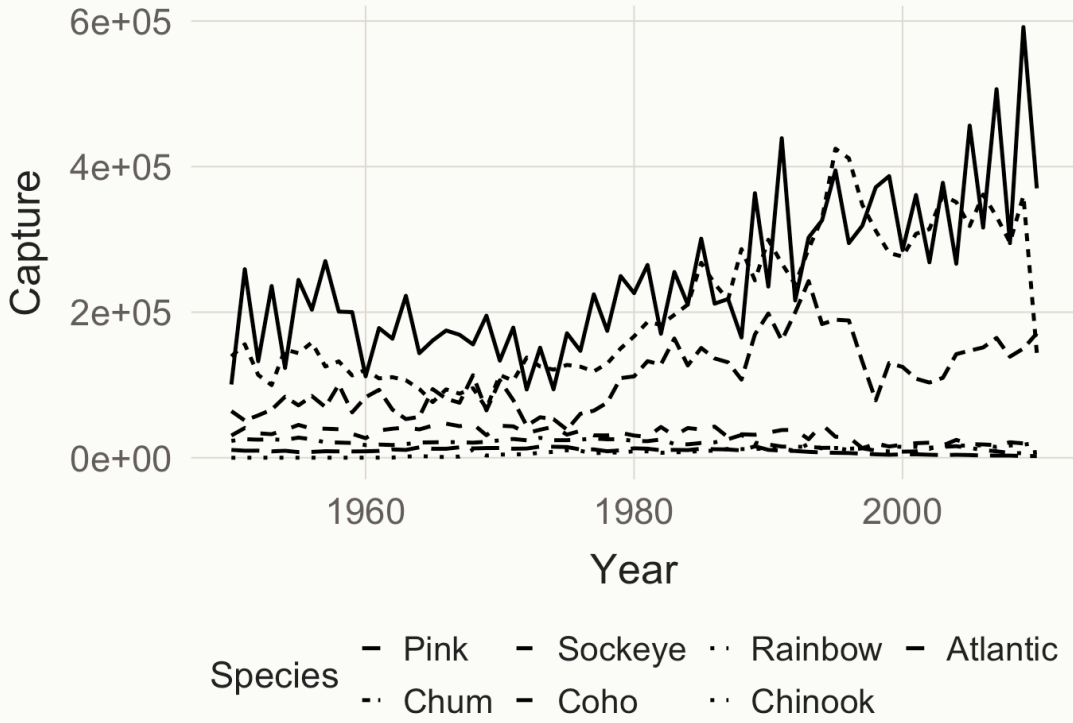
```
'data.frame': 427 obs. of  3 variables:
 $ Species: Factor w/ 7 levels "Pink","Chum",..: 1 1 1 1 1 1 1 1 1 1 ...
 $ Year   : int  1950 1951 1952 1953 1954 1955 1956 1957 1958 1959 ...
 $ Capture: int  100600 259000 132600 235900 123400 244400 203400 270119 200798 200085 ...
```

Мировой улов 7 видов лососёвых за 60 лет.

Эстетика linetype

```
ggplot(fish.tidy, aes(Year, Capture,
                      linetype = Species)) +
  geom_line()
```

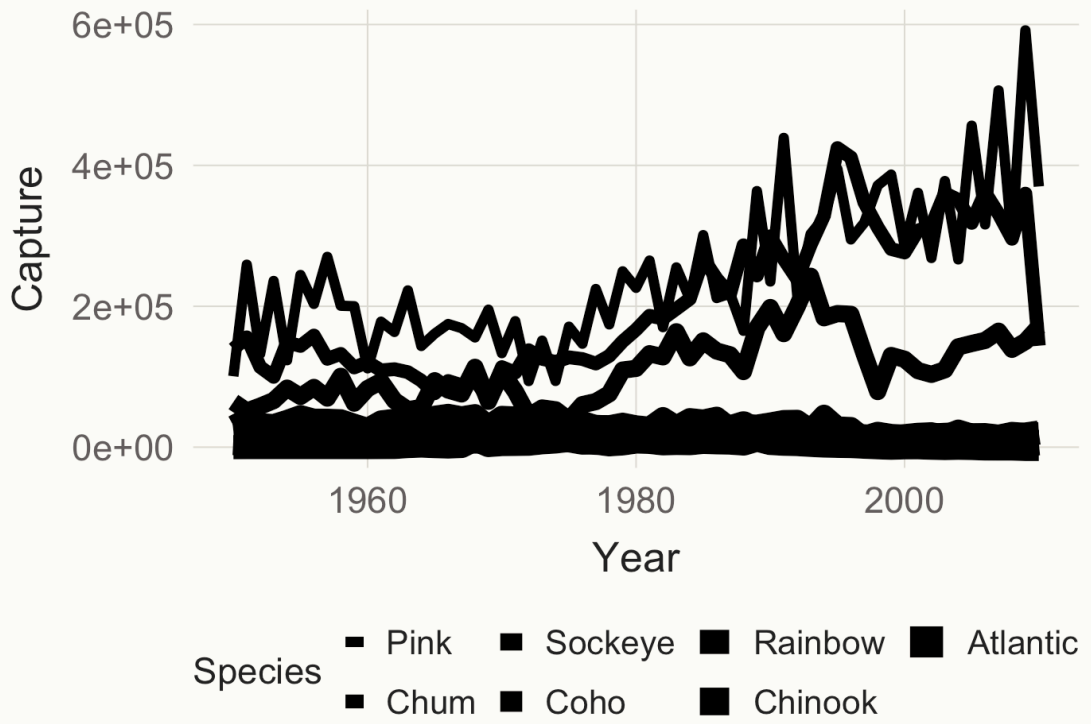
- Когда линий несколько, нужно подумать, какая эстетика лучше позволит различить отдельные тренды.
- С linetype отдельные виды различить трудно.



Эстетика size (linewidth)

```
ggplot(fish.tidy, aes(Year, Capture,
                      linewidth = Species)) +
  geom_line()
```

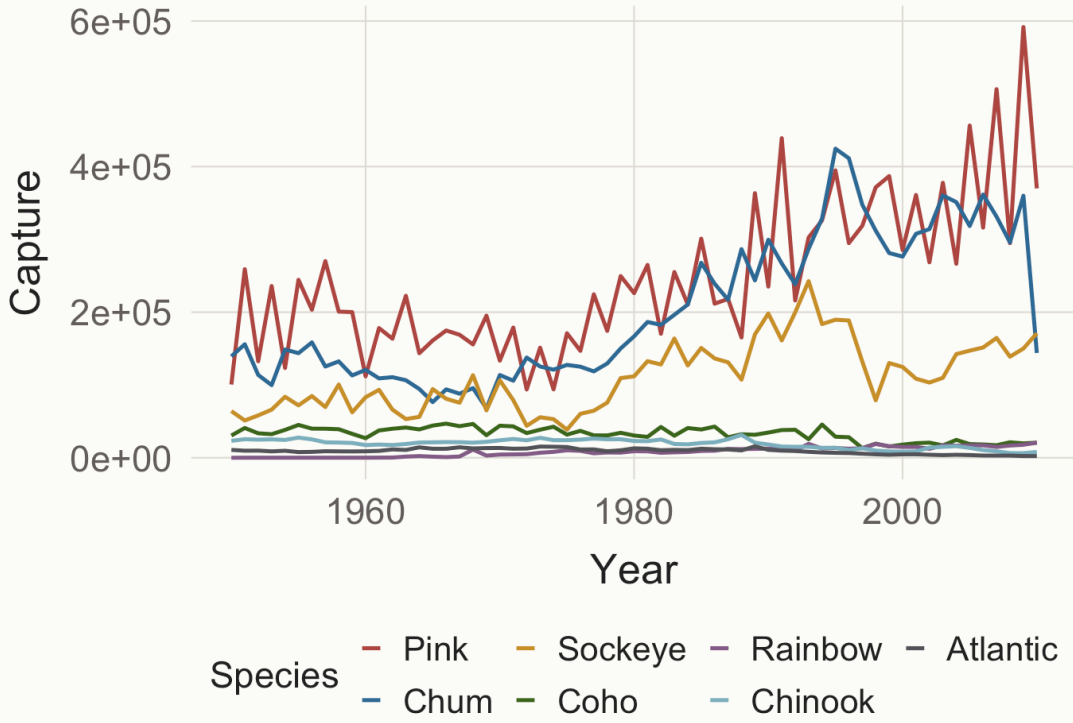
- Использовать толщину ещё хуже.



Эстетика color

```
ggplot(fish.tidy, aes(Year, Capture,
                      color = Species)) +
  geom_line()
```

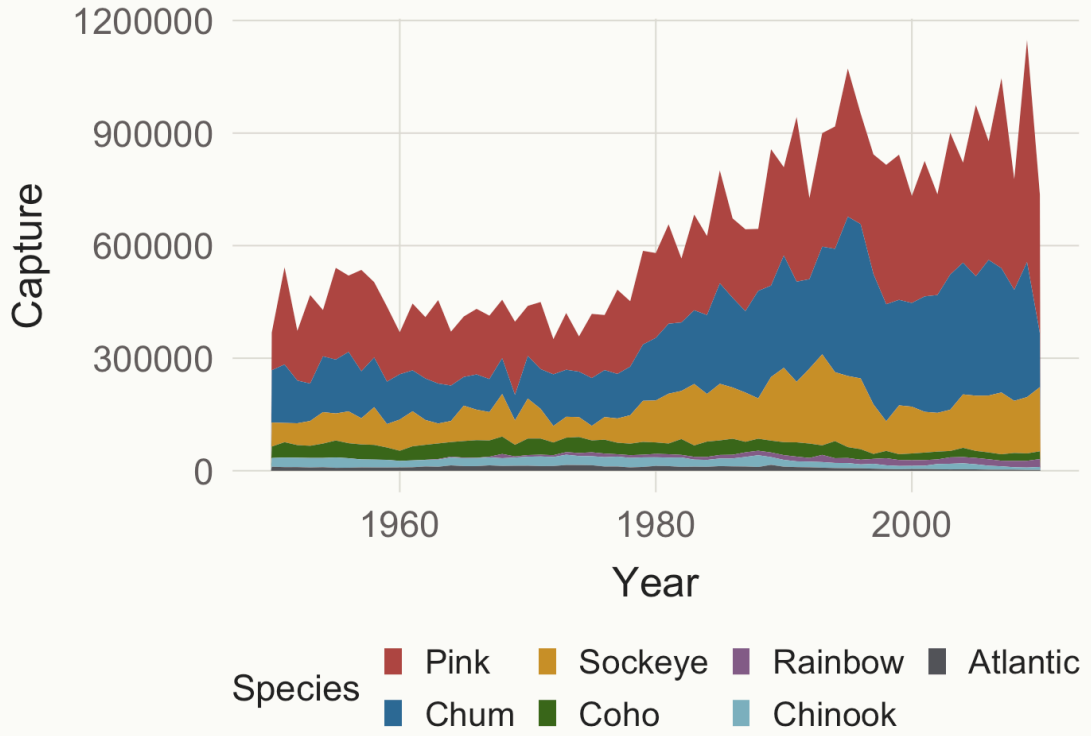
- Цвет позволяет легко различать группы.



Эстетика fill с `geom_area()`

```
ggplot(fish.tidy, aes(Year, Capture,
                      fill = Species)) +
  geom_area()
```

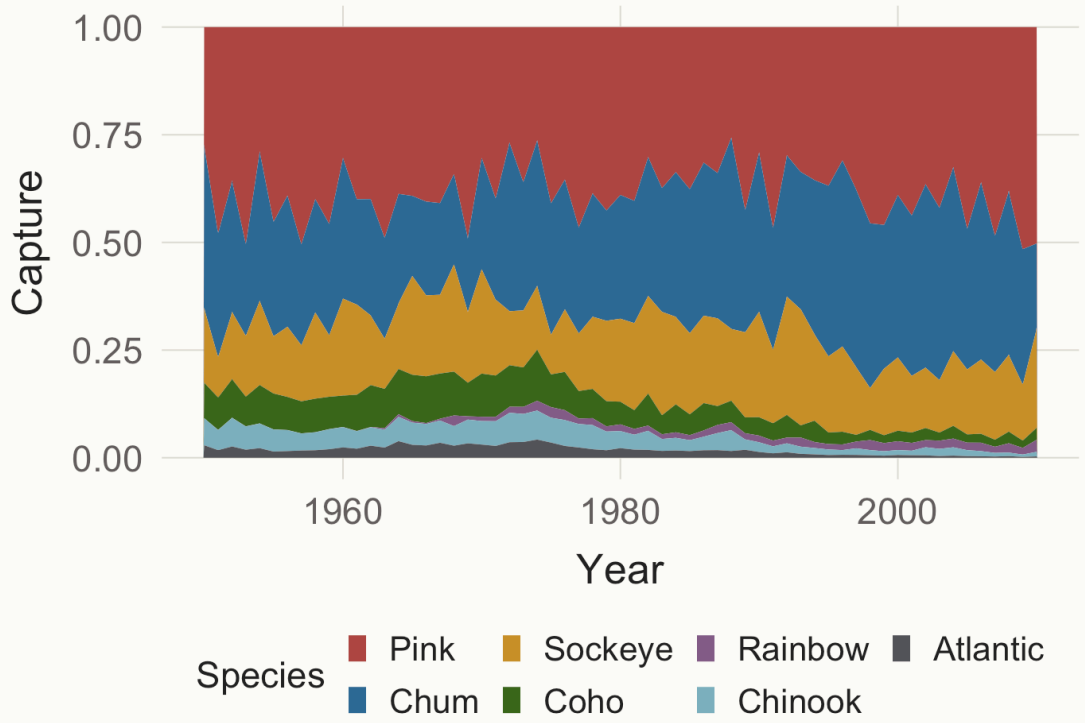
Можно использовать заливку областей через `geom_area`, у которой позиция по умолчанию — `"stack"`. Ряды не перекрываются, а складываются: в каждый момент времени значения ставятся друг на друга.



Использование position = "fill"

```
ggplot(fish.tidy, aes(Year, Capture,
                      fill = Species)) +
  geom_area(position = "fill")
```

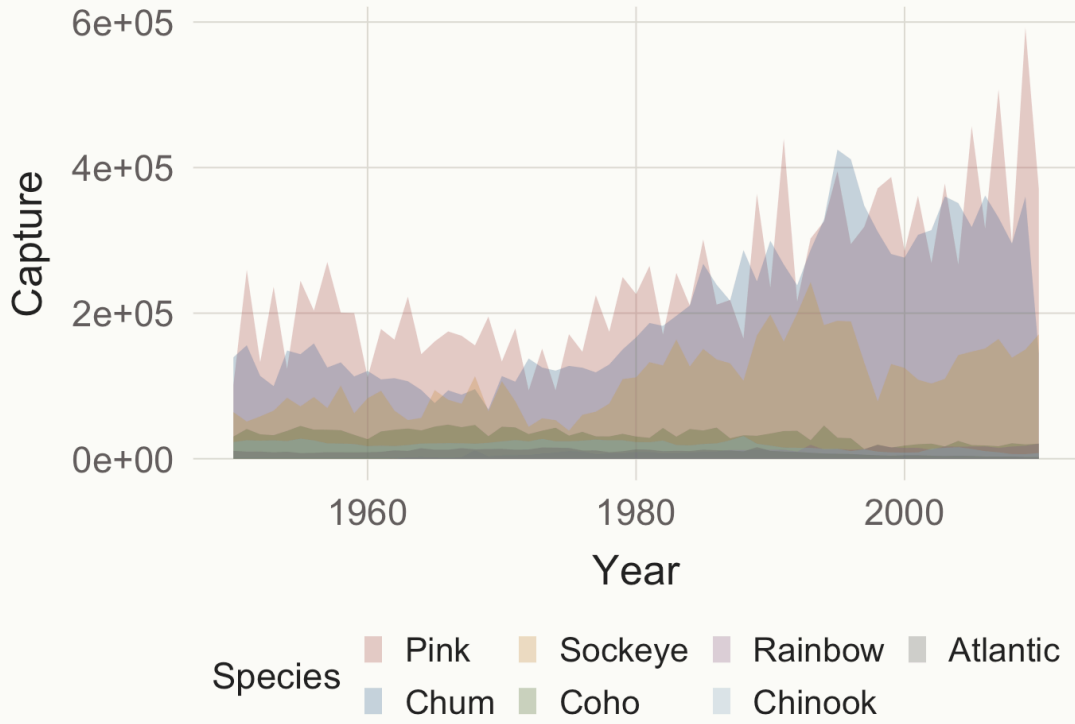
- Получим долю каждого вида в общем улове в каждый момент времени.
- Такие графики популярны для изучения изменения долей во времени.
- Трудность в том, что общая основа (0 или 1) есть только у нижней и верхней полос. Остальные «плавают», и их трудно сравнивать.



geom_ribbon()

Когда нужны перекрывающиеся области

```
ggplot(fish.tidy, aes(Year, Capture,
                      fill = Species)) +
  geom_ribbon(aes(ymax = Capture,
                  ymin = 0),
              alpha = 0.3)
```



Темы с нуля

Слой тем

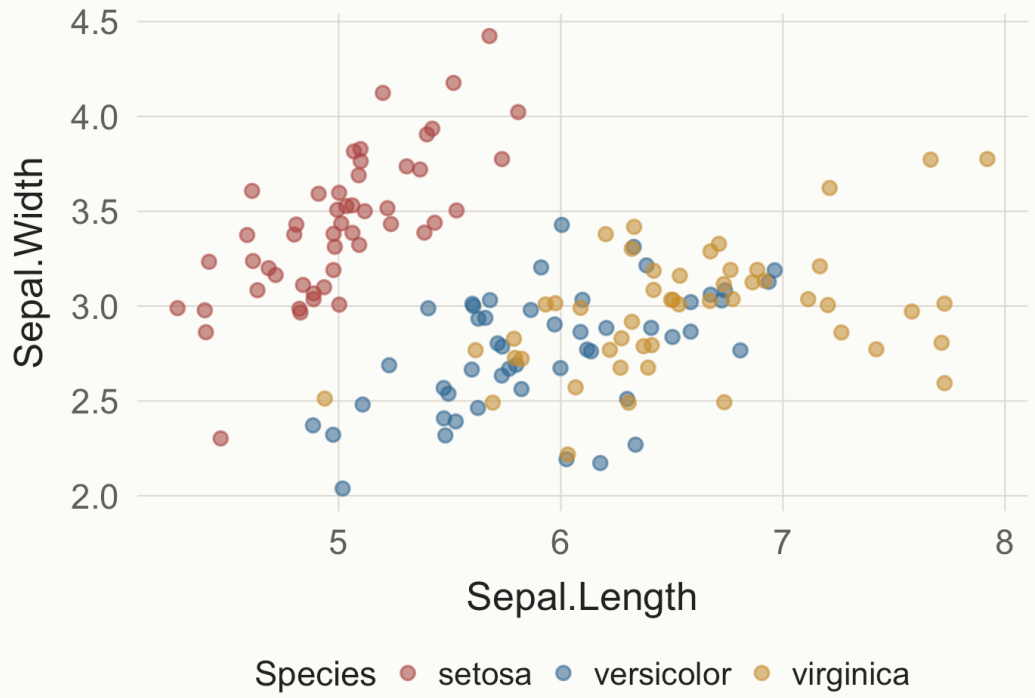
- Управляет всем оформлением графика, которое не относится к данным.
- Визуальные элементы, которые не являются частью данных.

Три типа:

Тип	Изменяется через
текст	`element_text()`
линия	`element_line()`
прямоугольник	`element_rect()`

Исходный график...

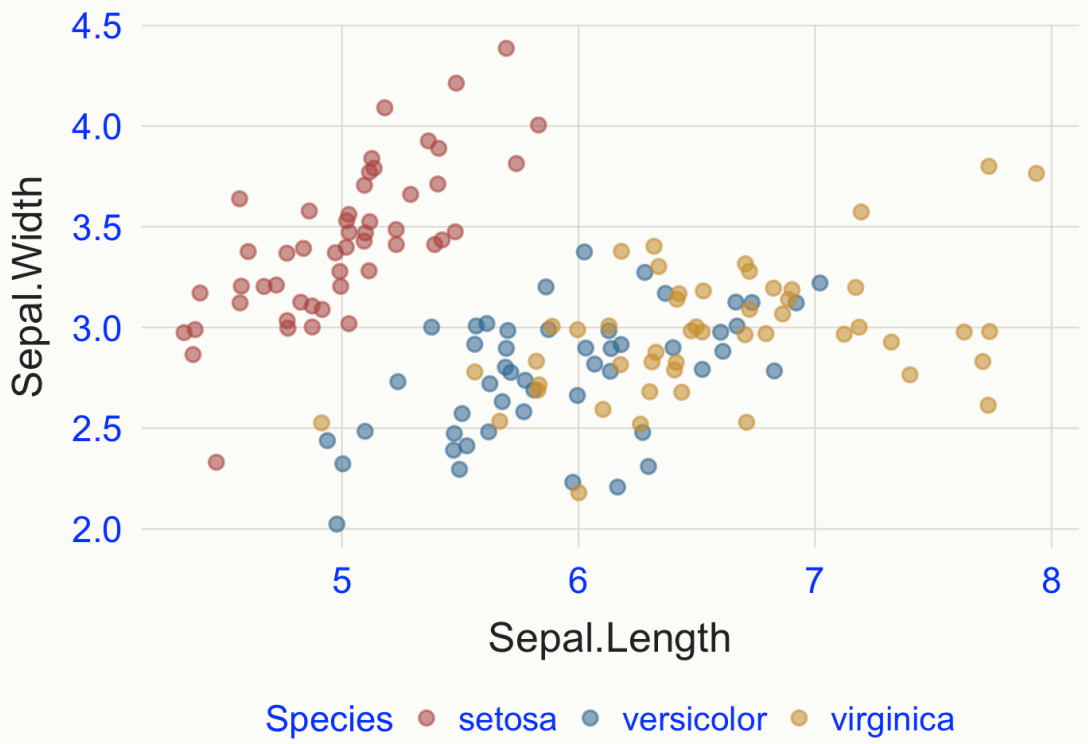
```
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = Species)) +
  geom_jitter(alpha = 0.6)
```



Состоит из элементов данных и оформления

Текстовые элементы

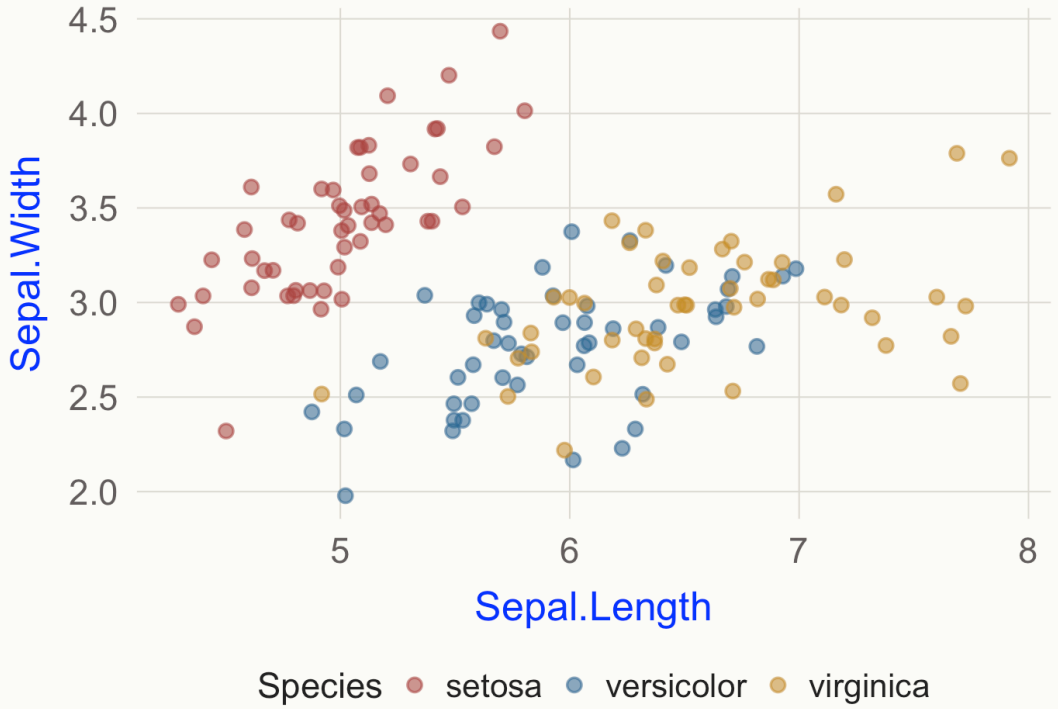
Все текстовые элементы нашего графика:



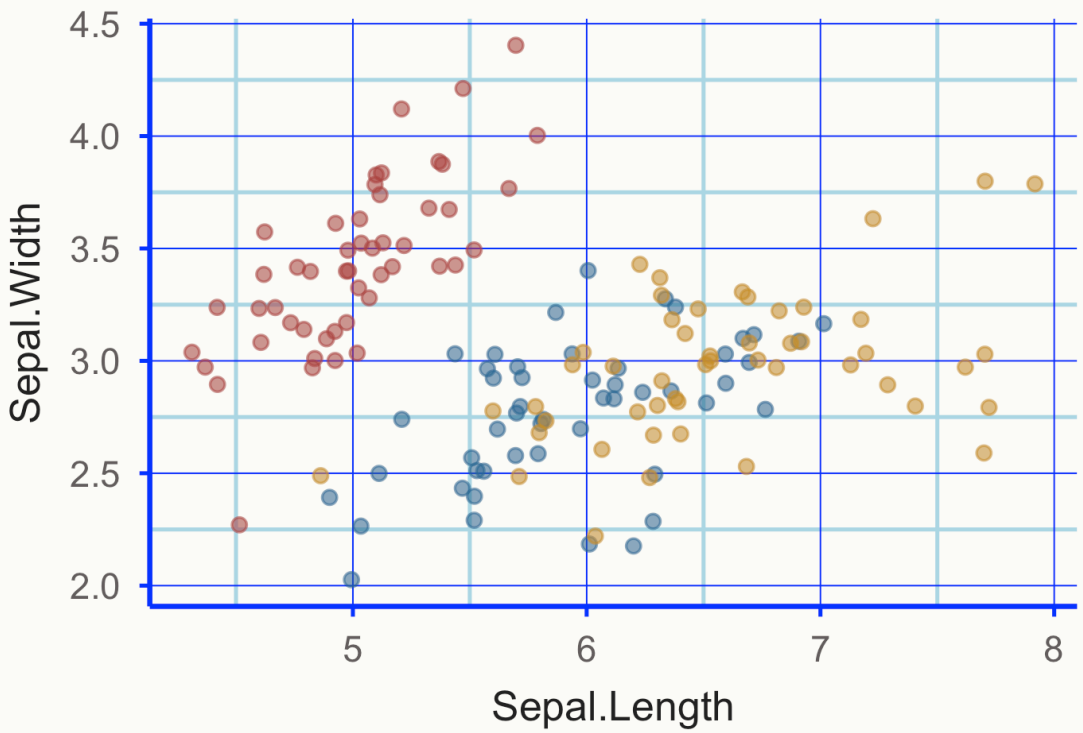

```
theme(
 text,
  axis.title,
    axis.title.x,
      axis.title.x.top,
      axis.title.x.bottom,
    axis.title.y,
      axis.title.y.left,
      axis.title.y.right,
  title,
    legend.title,
    plot.title,
    plot.subtitle,
    plot.caption,
    plot.tag,
  axis.text,
    axis.text.x,
      axis.text.x.top,
      axis.text.x.bottom,
    axis.text.y,
      axis.text.y.left,
      axis.text.y.right,
  legend.text,
  strip.text,
    strip.text.x,
    strip.text.y)
```

Настройка элементов темы

```
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = Species)) +
  geom_jitter(alpha = 0.6) +
  theme(axis.title = element_text(color = "blue"))
```

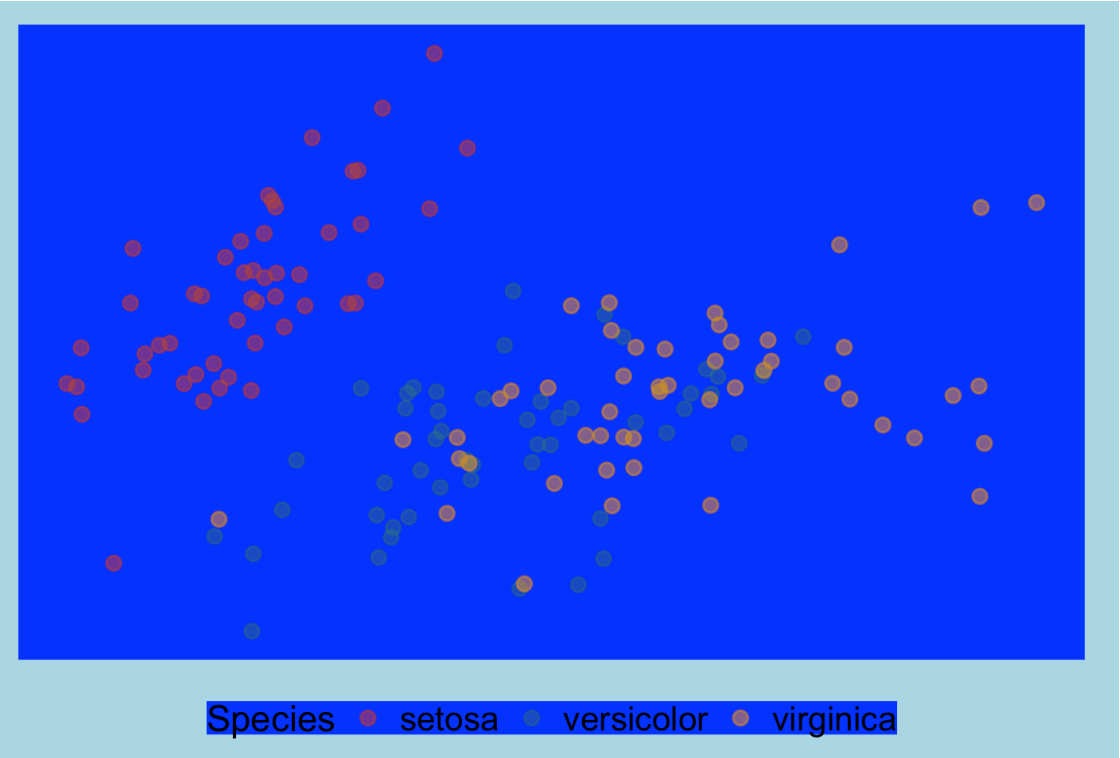


Линейные элементы




```
theme(
  line,
  axis.ticks,
    axis.ticks.x,
      axis.ticks.x.top,
      axis.ticks.x.bottom,
    axis.ticks.y,
      axis.ticks.y.left,
      axis.ticks.y.right,
  axis.line,
    axis.line.x,
      axis.line.x.top,
      axis.line.x.bottom,
    axis.line.y,
      axis.line.y.left,
      axis.line.y.right,
  panel.grid,
    panel.grid.major,
      panel.grid.major.x,
      panel.grid.major.y,
    panel.grid.minor,
      panel.grid.minor.x,
      panel.grid.minor.y
)
```

Прямоугольные элементы




```
theme(
  rect,
  legend.background,
  legend.key,
  legend.box.background,
  panel.background,
  panel.border,
  plot.background,
  strip.background,
    strip.background.x,
    strip.background.y
)
```

Иерархические имена отражают наследование

Хотя доступен каждый элемент, менять их по отдельности не нужно: они наследуют настройки друг от друга по иерархии.

Например, текст

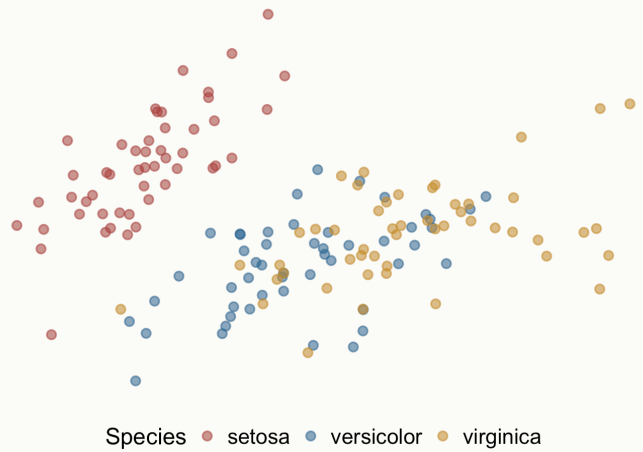
```
text
  axis.title
    axis.title.x
      axis.title.x.top
      axis.title.x.bottom
    axis.title.y
      axis.title.y.left
      axis.title.y.right
```

Например, линии

```
line
axis.ticks
  axis.ticks.x
    axis.ticks.x.top
    axis.ticks.x.bottom
  axis.ticks.y
    axis.ticks.y.left
    axis.ticks.y.right
axis.line
  axis.line.x
    axis.line.x.top
    axis.line.x.bottom
  axis.line.y
    axis.line.y.left
    axis.line.y.right
```

`element_blank()`

```
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = Species)) +
  geom_jitter(alpha = 0.6) +
  theme(line = element_blank(),
        rect = element_blank(),
        text = element_blank())
```



`element_blank()` удаляет элемент. Обратите внимание: значки в легенде (ключи) относятся к данным, поэтому остаются.

В ggplot2 4.x подписи легенды остаются — их убирают через `legend.title` и `legend.text`.

Способы использования тем

1. С нуля
2. Тема, сохранённая как объект
3. Встроенные темы
    - пакеты ggplot2 или ggthemes
4. Встроенные темы из других пакетов
5. Обновление и установка темы по умолчанию

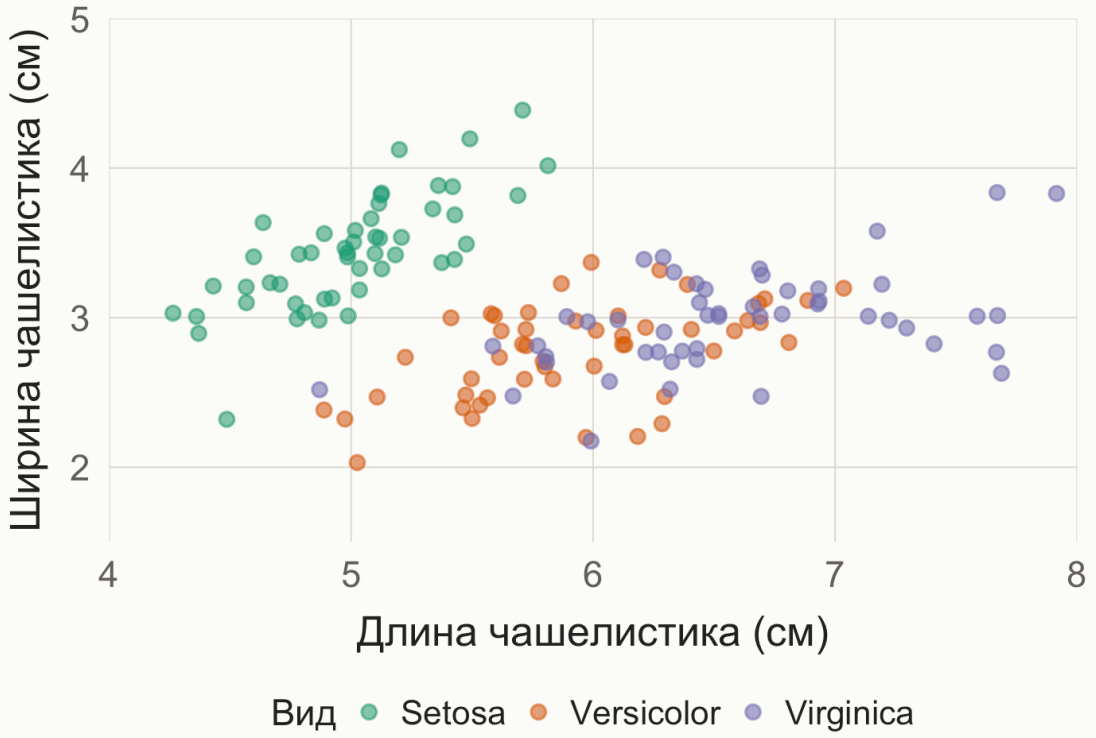
Определение объектов темы

- Удобно, когда графиков много
- Обеспечивает единый стиль
- Одна тема применяется везде.

Создание темы с нуля — подробный процесс, который не хочется повторять для каждого графика.

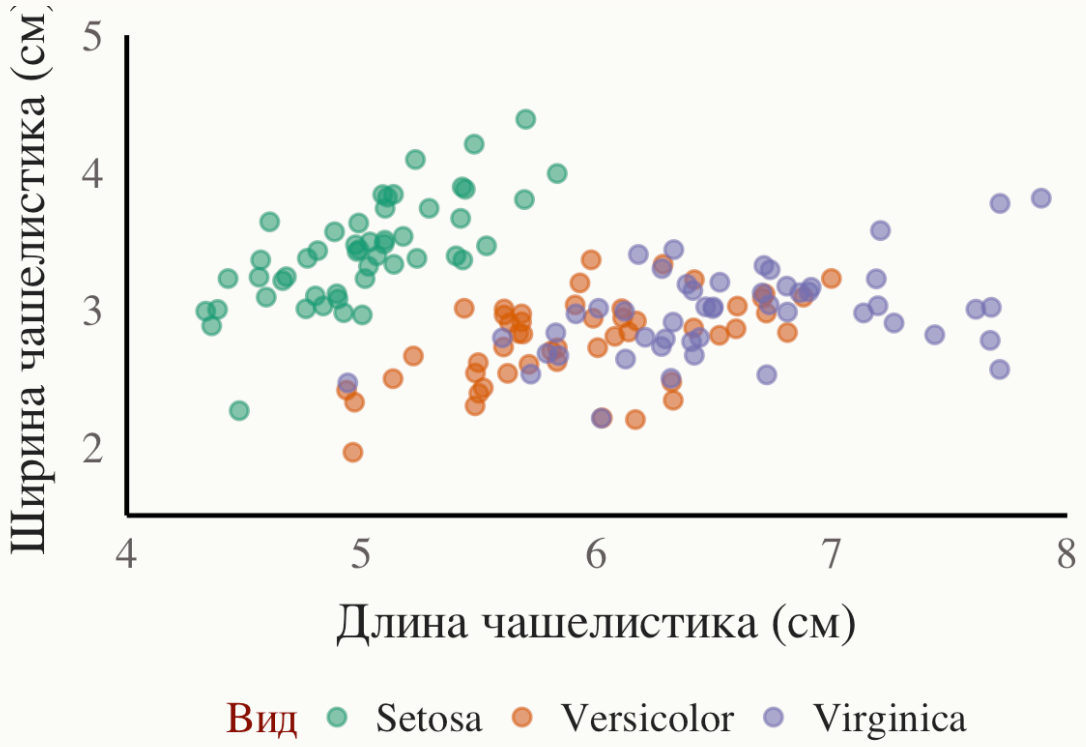
Определение объектов темы

```
z <- ggplot(iris,
  aes(x = Sepal.Length,
      y = Sepal.Width,
      color = Species)) +
  geom_jitter(alpha = 0.6) +
  scale_x_continuous(
    "Длина чашелистика (см)",
    limits = c(4, 8),
    expand = c(0, 0)) +
  scale_y_continuous(
    "Ширина чашелистика (см)",
    limits = c(1.5, 5),
    expand = c(0, 0)) +
  scale_color_brewer(
    "Вид",
    palette = "Dark2",
    labels = c(
      "Setosa",
      "Versicolor",
      "Virginica"))
z
```



Настройка отдельных аргументов темы

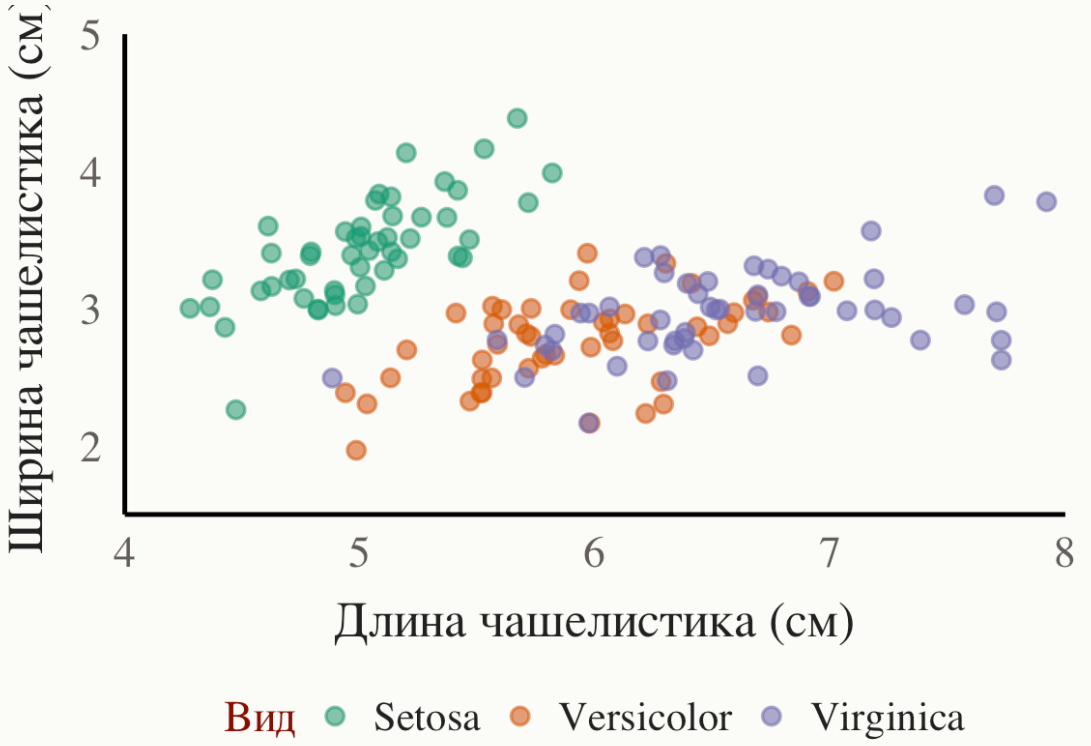
```
z + theme(
  text = element_text(family = "serif",
                      size = 14),
  rect = element_blank(),
  panel.grid = element_blank(),
  title = element_text(color = "#8b0000"),
  axis.line = element_line(color = "black")
)
```



Сохраняем тему как объект

```
theme_iris <- theme(
  text = element_text(family = "serif",
                      size = 14),
  rect = element_blank(),
  panel.grid = element_blank(),
  title = element_text(color = "#8b0000"),
  axis.line = element_line(color = "black")
)

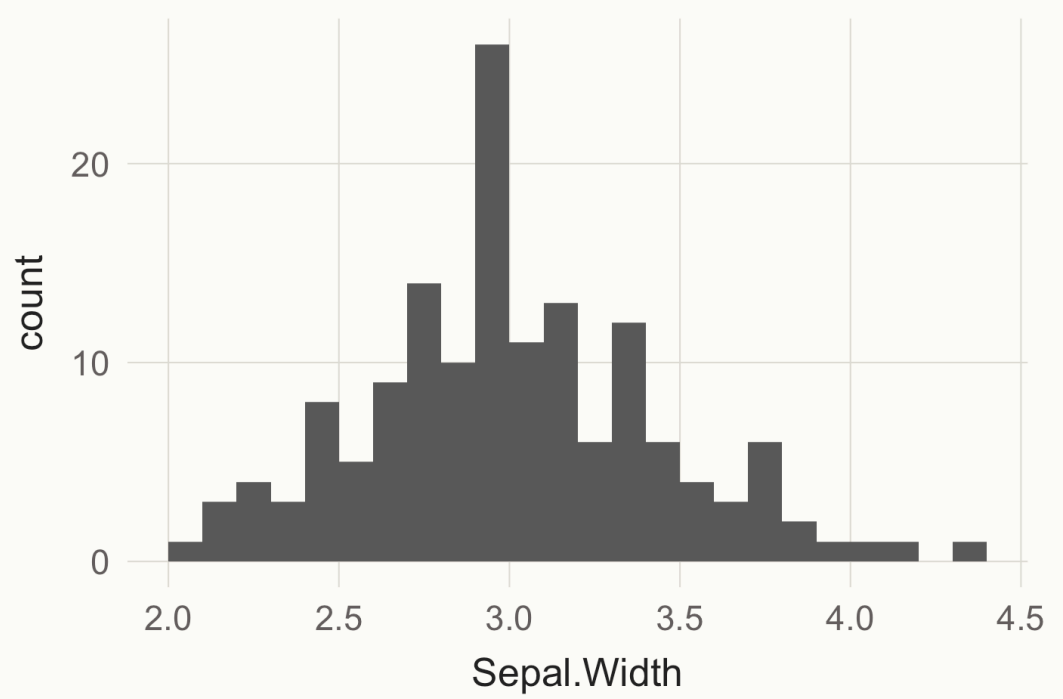
z + theme_iris
```



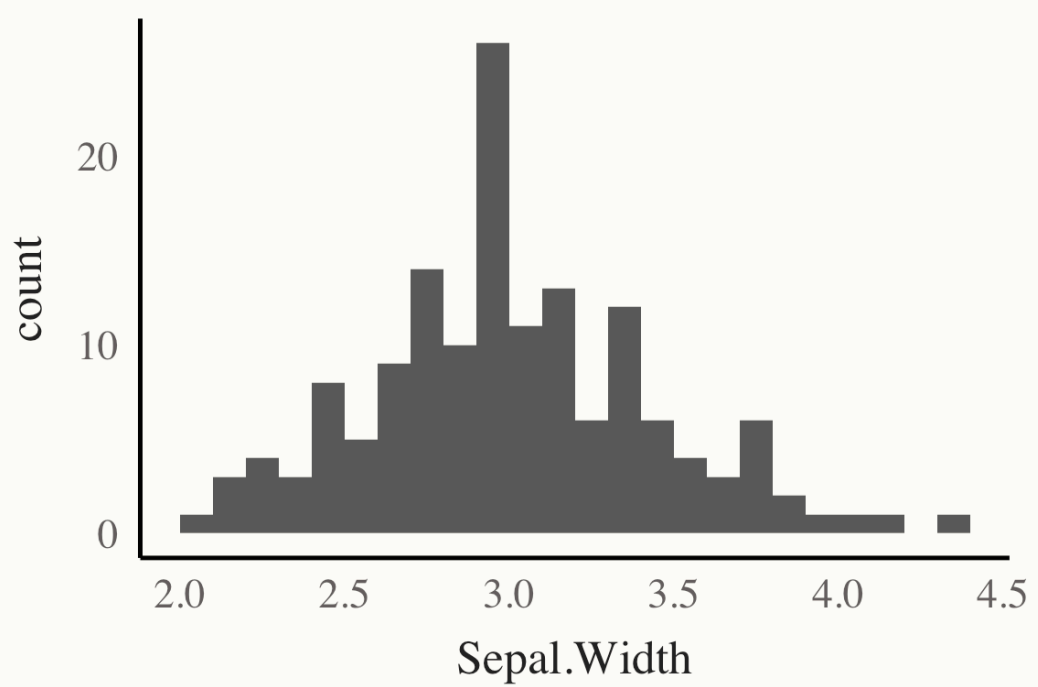
Повторное использование объектов темы

```
m <- ggplot(iris, aes(x = Sepal.Width)) +
  geom_histogram(binwidth = 0.1, center = 0.05)

m
```

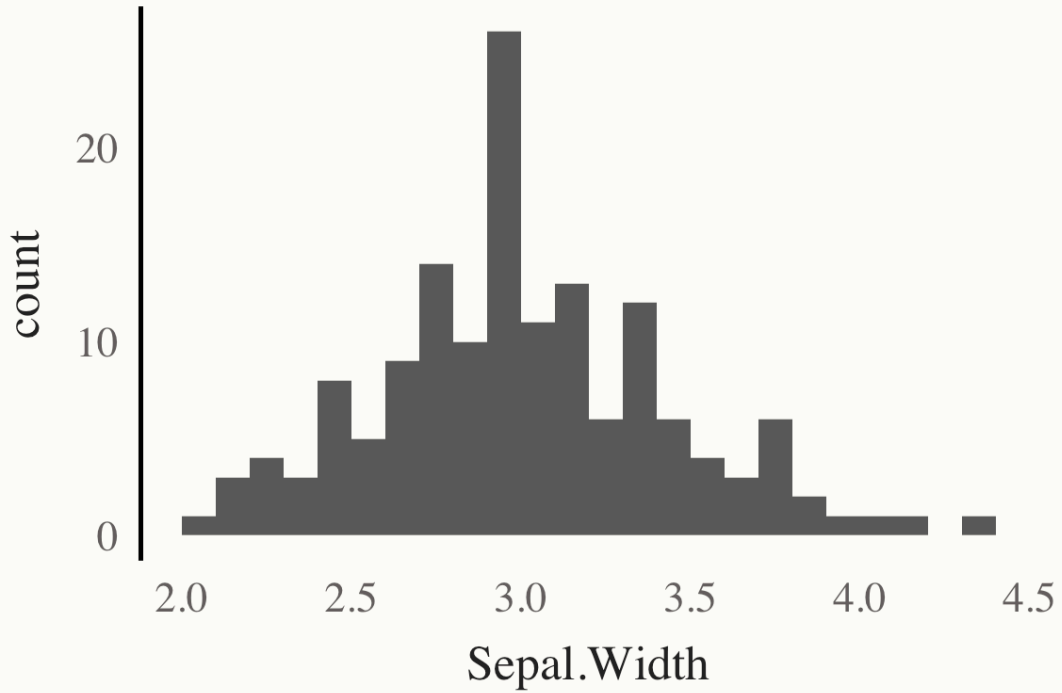



```
m +
  theme_iris
```



Переопределение прежних настроек

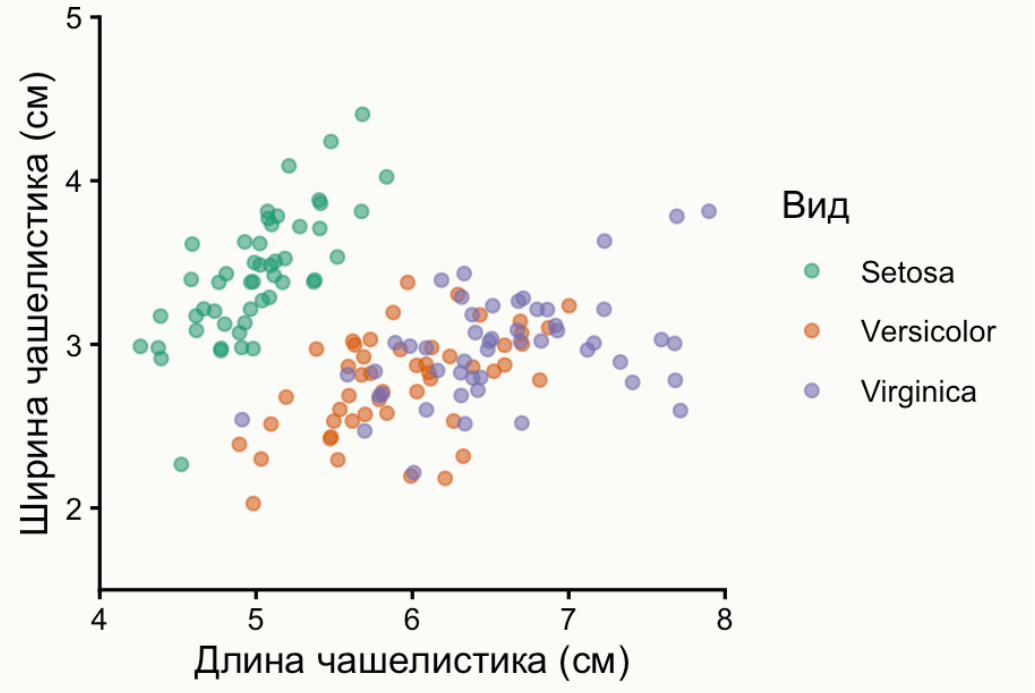
```
m +
  theme_iris +
  theme(axis.line.x = element_blank())
```



Встроенные темы

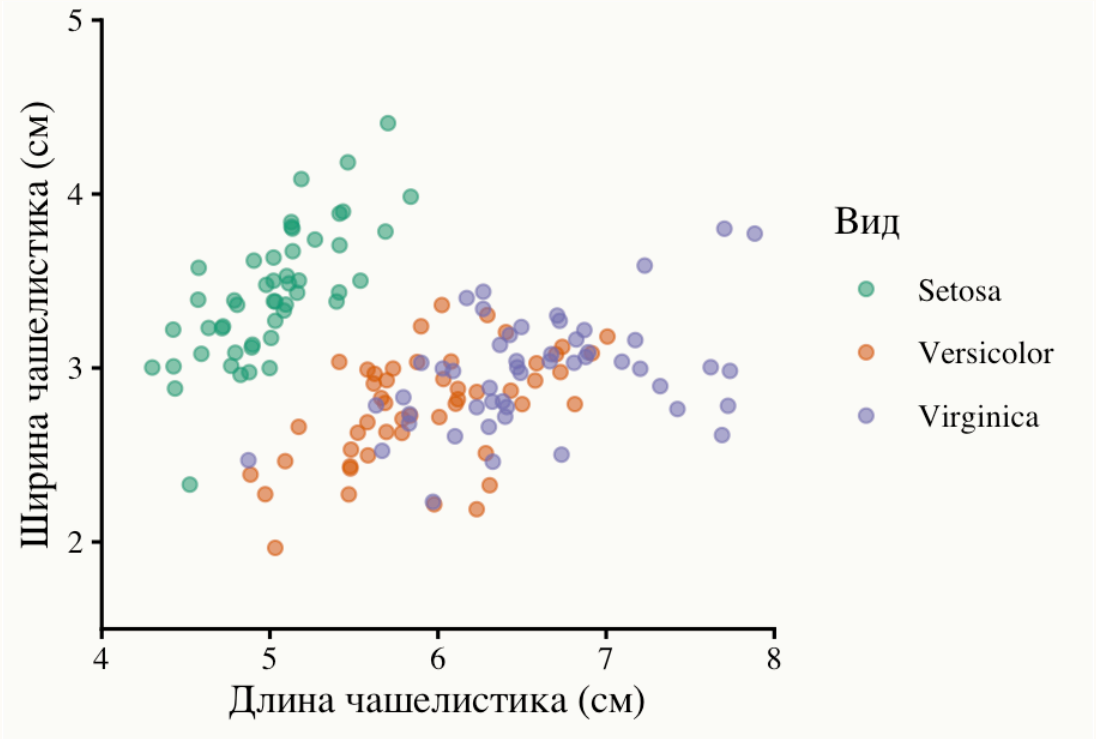
Для встроенных тем используйте функции `theme_*()`.

```
z +
  theme_classic()
```



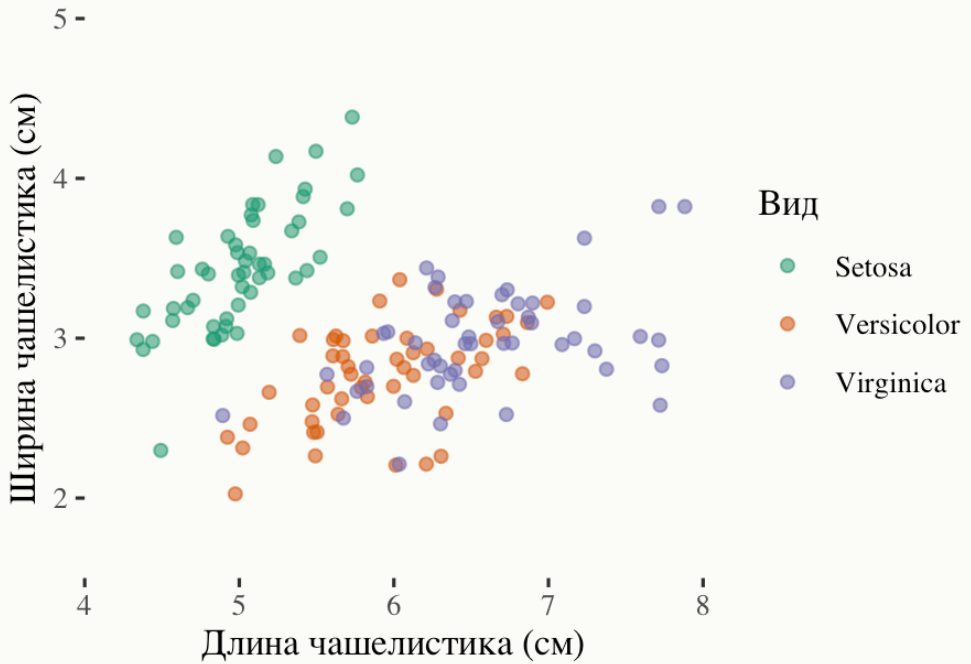
Любой элемент всегда можно изменить

```
z +
  theme_classic() +
  theme(text = element_text(family = "serif"))
```



Пакет ggthemes

```
library(ggthemes)
z +
  theme_tufte()
```



Убирает всё оформление, не относящееся к данным, и задаёт шрифт с засечками.

Обновление тем

Обновить тему по умолчанию (глобально):

```
theme_update(
  text = element_text(family = "serif", size = 14),
  rect = element_blank(),
  panel.grid = element_blank(),
  title = element_text(color = "#8b0000"),
  axis.line = element_line(color = "black")
)
```

Установить тему по умолчанию:

```
theme_set(theme_grey())  # Вернуть стандартную тему ggplot2
```